MACHINE LEARNING / MODULE 4

Dimensionality reduction and feature selection

Choosing, compressing and mapping many correlated measurements

Full day · 09:00–17:00 | 6 parts | 2 laboratories

Machine learning for energy applications

Today's plan

09:00	Welcome, the curse of dimensionality, feature selection	Lecture
10:30	Break	Break
10:45	PCA in depth	Lecture
11:40	Lab 1: PCA on drilling data	Lab
12:40	Lunch	Lunch
13:40	Lab 1 debrief; kernel PCA, LDA, ICA and NMF	Lecture
14:25	Manifold learning: Isomap, LLE, t-SNE and UMAP	Lecture
15:15	Break	Break
15:30	Lab 2: nonlinear maps and a fair comparison	Lab
16:30	Debrief, reduction in practice and exit check	Discussion

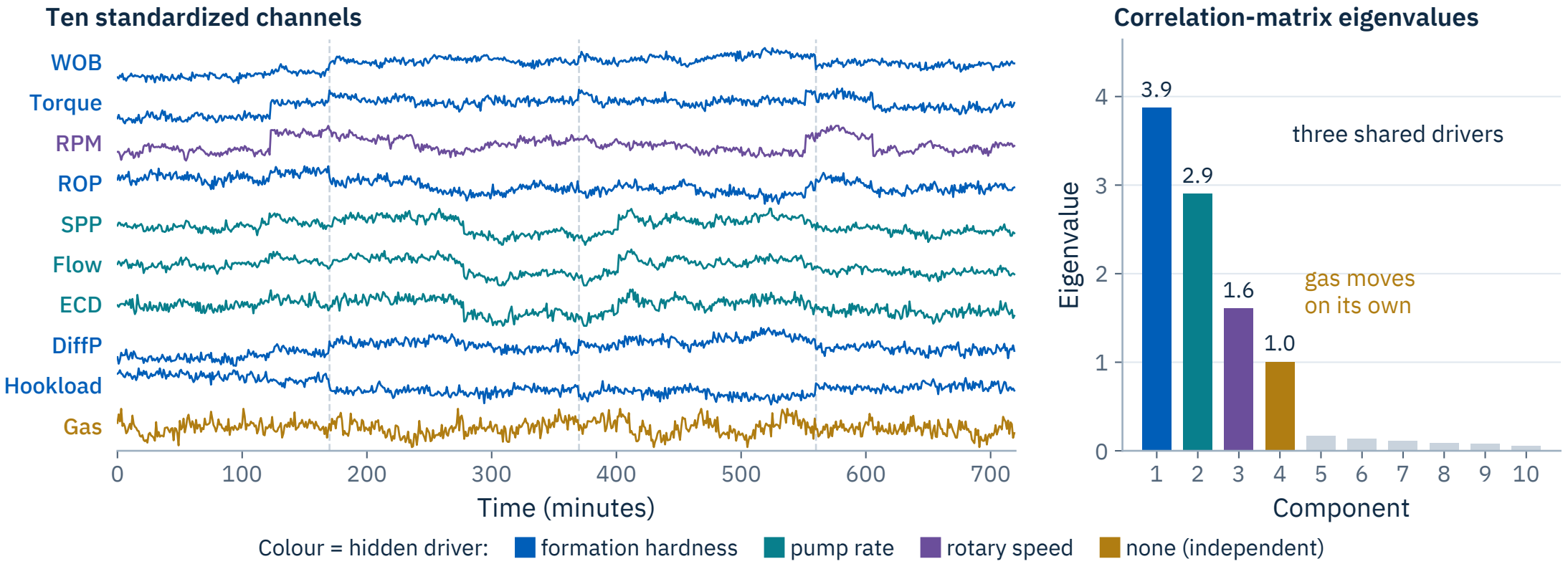
What you will be able to do

By the end of the day you will be able to:

1. Explain why many features weaken distance-based methods, and when to reduce.
2. Select features with filter, wrapper and embedded methods inside cross-validation.
3. Compute and validate a PCA: scaling, loadings, number of components, reconstruction error.
4. Choose kernel PCA, LDA, ICA or NMF for a stated goal.
5. Build t-SNE, Isomap and UMAP maps and state what they cannot show.
6. Judge a representation by trustworthiness, stability and held-out task score.

Ten drilling channels move as a few groups

Twelve hours of rig data: how many numbers per minute do you really need?



Synthetic data: three latent drivers plus noise (illustrative, seed 4). Right: eigenvalues, coloured by the driver each component follows.

PART 1 OF 6

01

Why reduce dimensions

More columns add noise and empty space before information.

- The curse of dimensionality in numbers
- Irrelevant features and nearest neighbours
- Selection versus extraction
- What each method preserves

Reduction serves four different goals

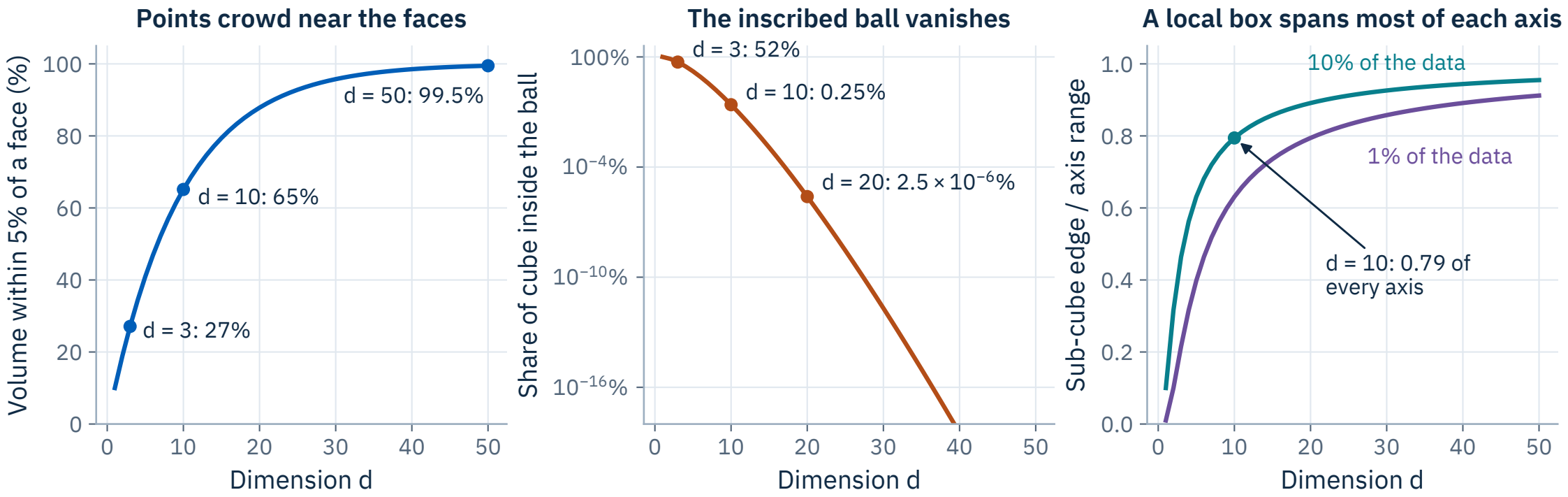
Goal	Useful output	Evidence to inspect
Visualization	Two- or three-dimensional coordinates	Neighbourhood preservation
Compression	Lower-dimensional representation	Reconstruction error
Denoising	Restricted representation	Signal preservation
Prediction	Features for a downstream estimator	Held-out task performance

Feature selection keeps original variables; feature extraction builds new ones.

◆ Name the goal before choosing a method: the goal decides which evidence counts.

High-dimensional volume concentrates near the edges

Three exact calculations for points spread uniformly in a unit cube.



Exact formulas: shell volume 1 − 0.9^d; inscribed-ball share π^(d/2) / Γ(d/2 + 1) × 0.5^d; sub-cube edge r^(1/d) for a fraction r of the data.

Keeping neighbour density needs exponentially more data

To keep 10 samples along every axis, a regular grid needs 10^d points:

10

samples for 1 feature

100,000

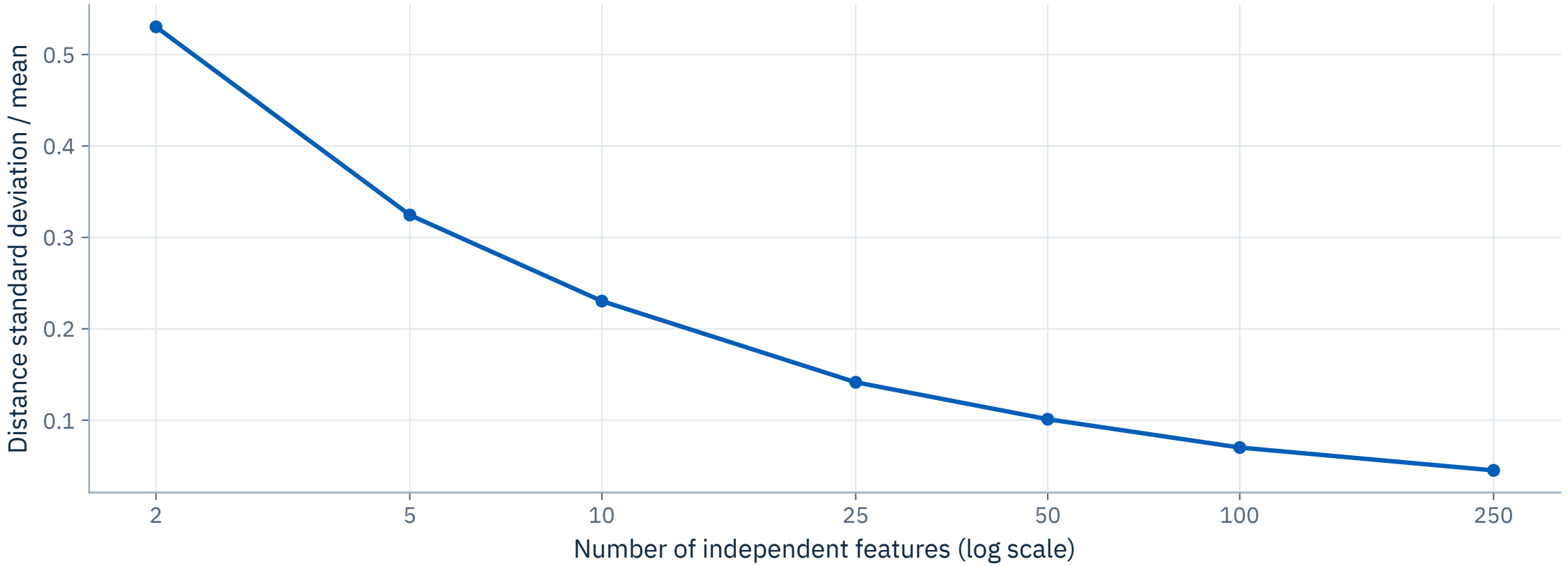
samples for 5 features

10 billion

samples for 10 features

- Real data escape this only because measurements share a few drivers and lie near lower-dimensional structure.

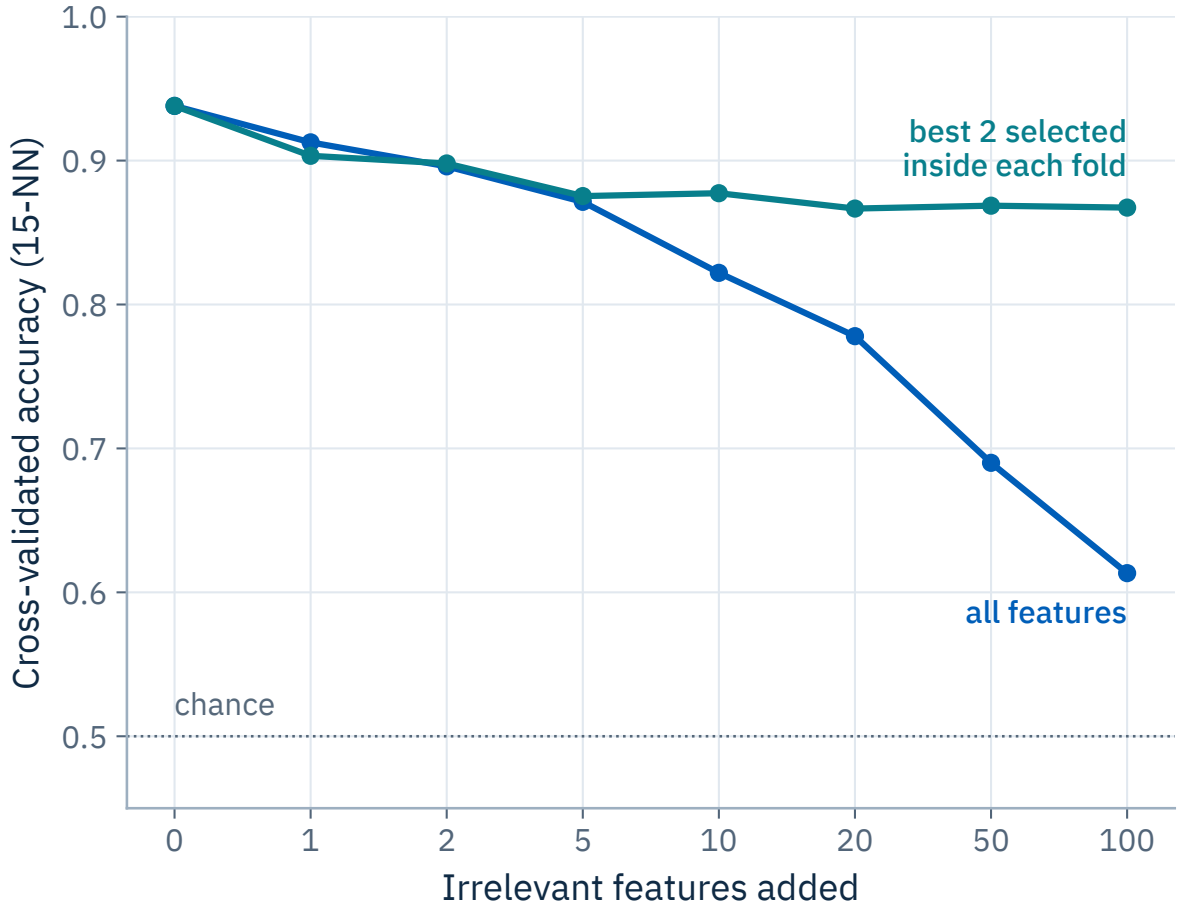
Distances can become less distinctive



Simulation with independent standard-normal features. Relative pairwise-distance spread shrinks as dimension grows.

Irrelevant features drown a nearest-neighbour signal

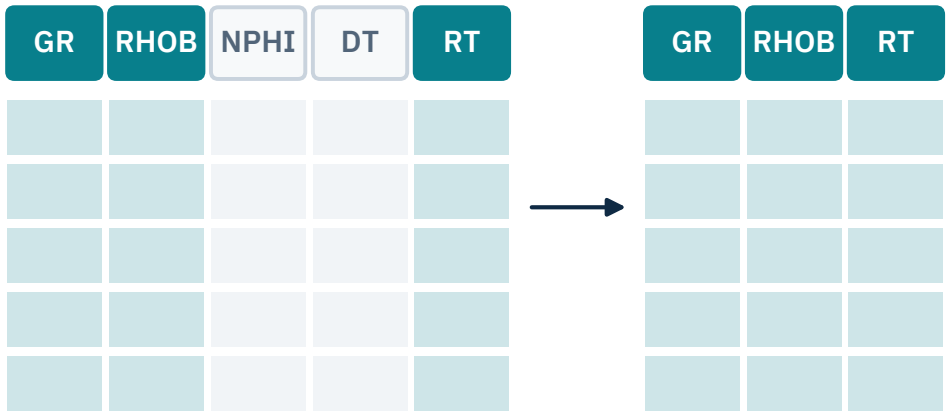
- Two informative features; pure-noise columns added in blocks
- 15-NN accuracy falls from 0.94 to 0.61 with 100 noise columns
- Selecting the best 2 features inside each fold holds 0.87



Synthetic two-class data (seeds 0–4); 5-fold CV averaged over five datasets.

Selection keeps columns; extraction builds new ones

Feature selection

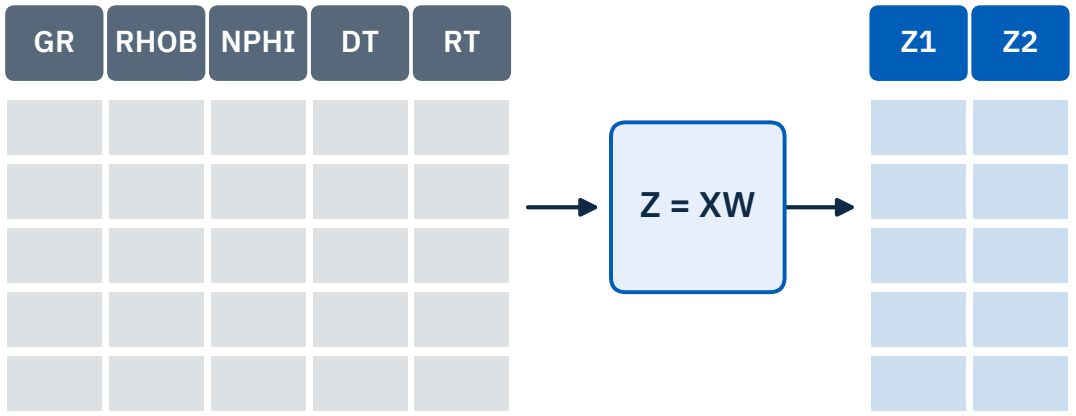


keeps names and units

unused sensors can go

A subset of the original columns

Feature extraction



every column contributes

new axes need interpretation

New columns built from all of them

Schematic. Selection returns a subset of the original logs; extraction returns new variables such as principal-component scores.

Reduction methods preserve different things

Method	Preserves	Uses labels	New observations
PCA, kernel PCA	Variance, linear or in a kernel space	No	`transform()`
LDA	Separation of known classes	Yes	`transform()`
Isomap, LLE	Geodesic or local linear structure	No	Approximate `transform()`
t-SNE	Local neighbourhoods	No	None in sklearn TSNE
UMAP	Neighbourhood graph	Optional	Fitted `transform()`

Parameter settings and preprocessing remain part of the method.

PRACTICE

4 min · Pairs

Match each energy task to a reduction goal

For each task, name the goal, choose selection or extraction, and name the evidence you would inspect:

1. Plot 25 wells' log suites to spot tool problems before interpretation
2. Store 1 Hz rig data in fewer channels, reconstructible within a tolerance
3. Predict ROP from 60 candidate surface and mud features
4. Decide which of 12 redundant sensors to keep on the next rig

PART 2 OF 6

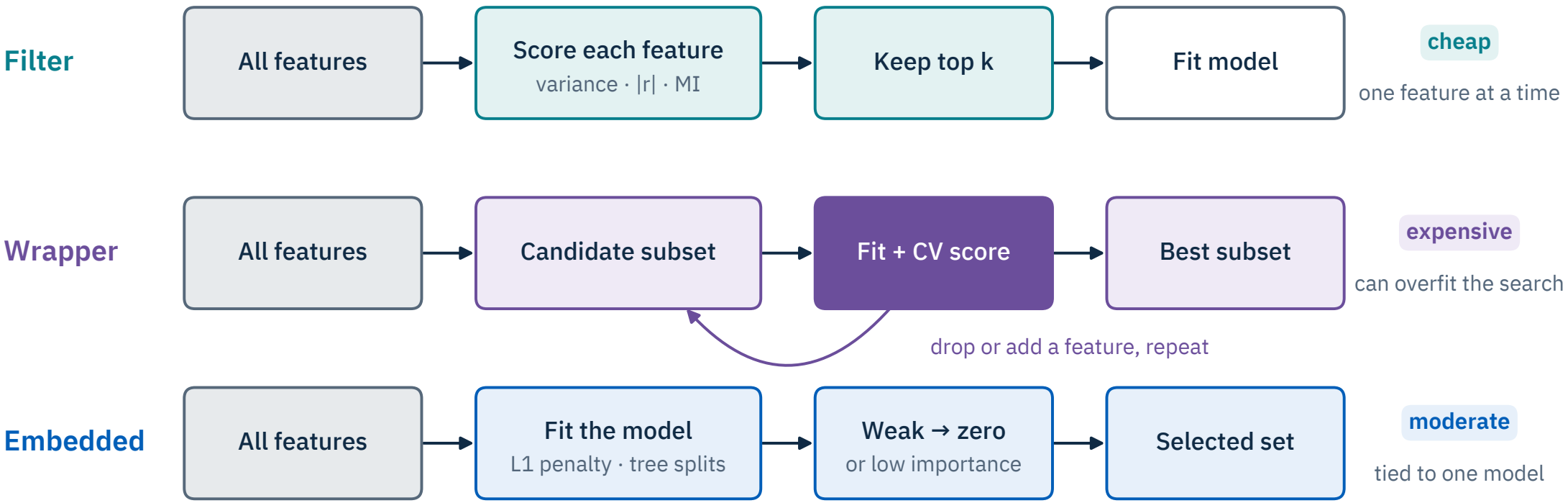
02

Feature selection

Keep the columns that matter; prove it on held-out folds.

- Filters: variance, redundancy, mutual information
- Wrappers: recursive feature elimination
- Embedded: L1 penalties and tree importance
- Leakage and the stability of selected sets

Three selection families use the model differently



Schematic. Filters score features without the final model; wrappers search subsets with it; embedded methods select while fitting it.

Filters rank features before any model is fitted

Variance threshold

- Drops constant or near-constant channels
- Scale-dependent: set it per sensor, in raw units

Redundancy

- Flags pairs of features with high $|r|$
- Keep the cheaper or more reliable sensor

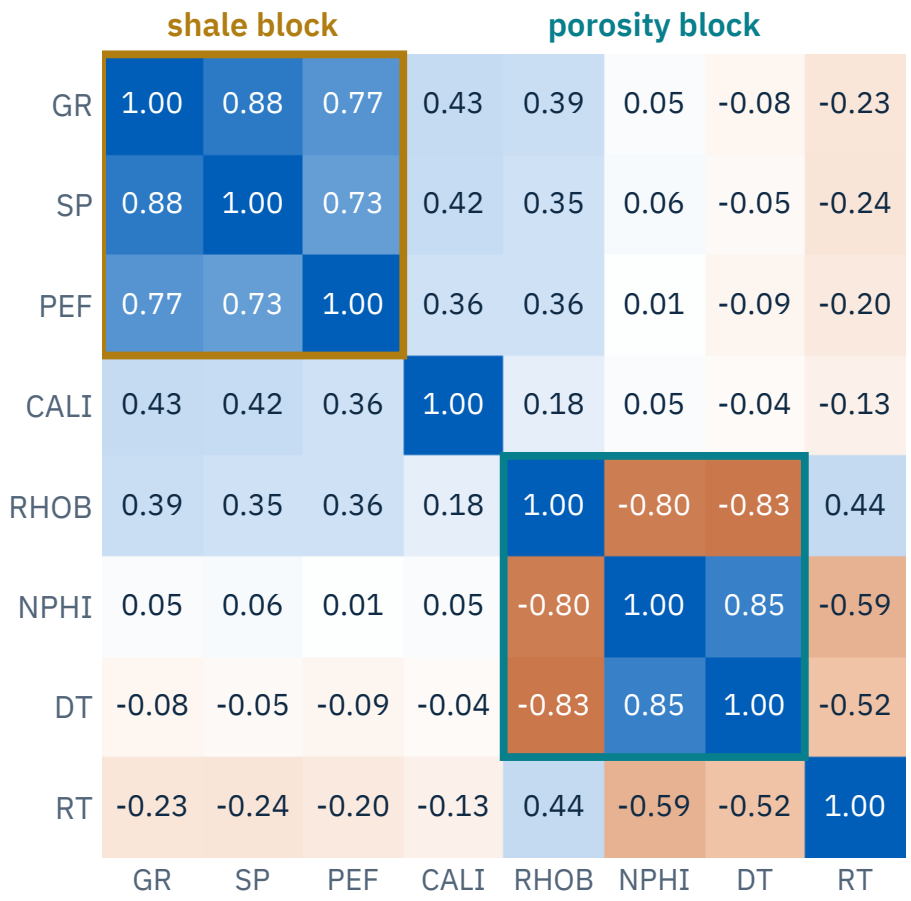
Relevance

- Scores each feature against the target
- F-test for linear links, mutual information for any

◆ Filters are fast and model-free, but they judge one feature at a time.

Correlated logs form redundant blocks

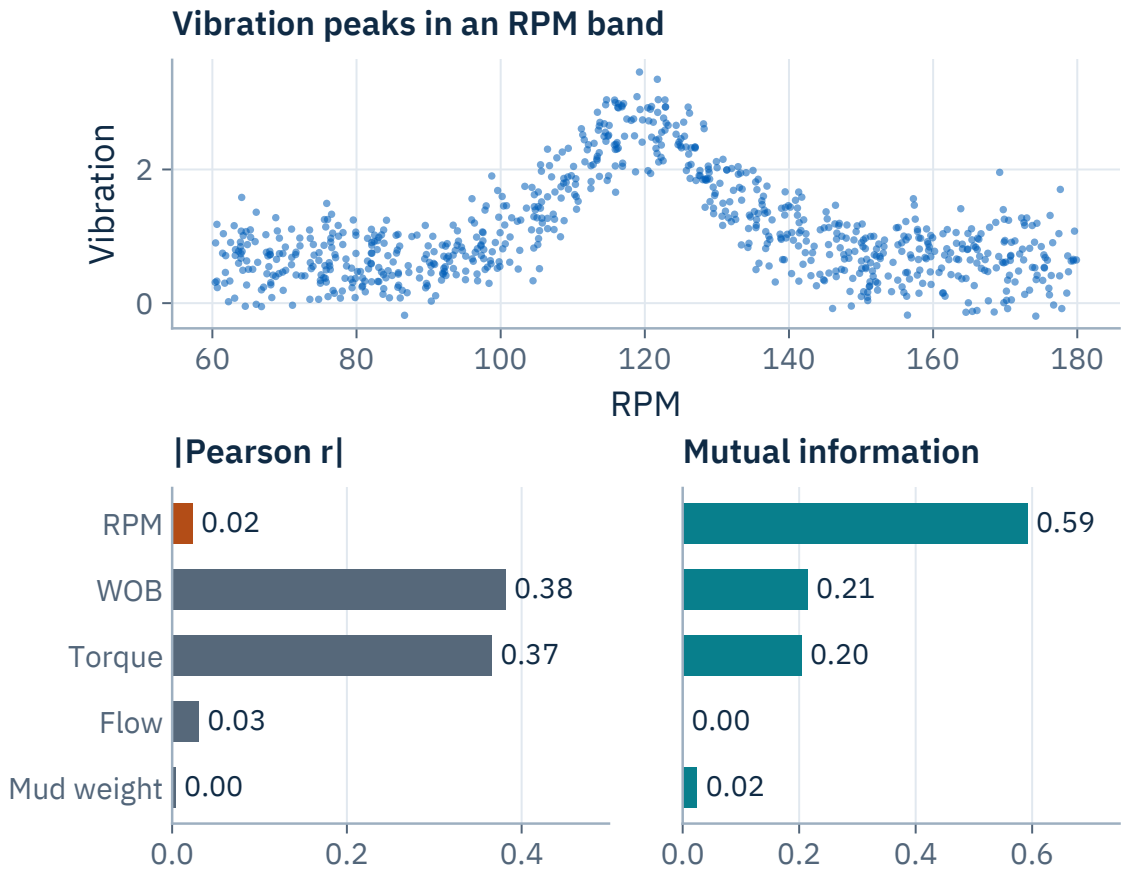
- GR, SP and PEF: $|r| \geq 0.73$, one shale signal
- RHOB, NPHI and DT: $|r| \geq 0.80$, one porosity signal
- Keep one log per block, or summarize the block with PCA
- Pearson r sees only linear, pairwise links



Synthetic log suite (seed 11); orange cells are negative correlations.

Mutual information catches nonlinear relevance

- Vibration peaks in an RPM band, yet $|r|$ for RPM is only 0.02
- Mutual information ranks RPM first (0.59): it scores any dependence
- WOB and torque score alike because they carry one signal
- Still one feature at a time: blind to redundancy and interactions



Synthetic example (seed 7): resonance peak plus a weak WOB effect. MI from the k-nearest-neighbour estimator.

Recursive elimination drops the weakest feature each round

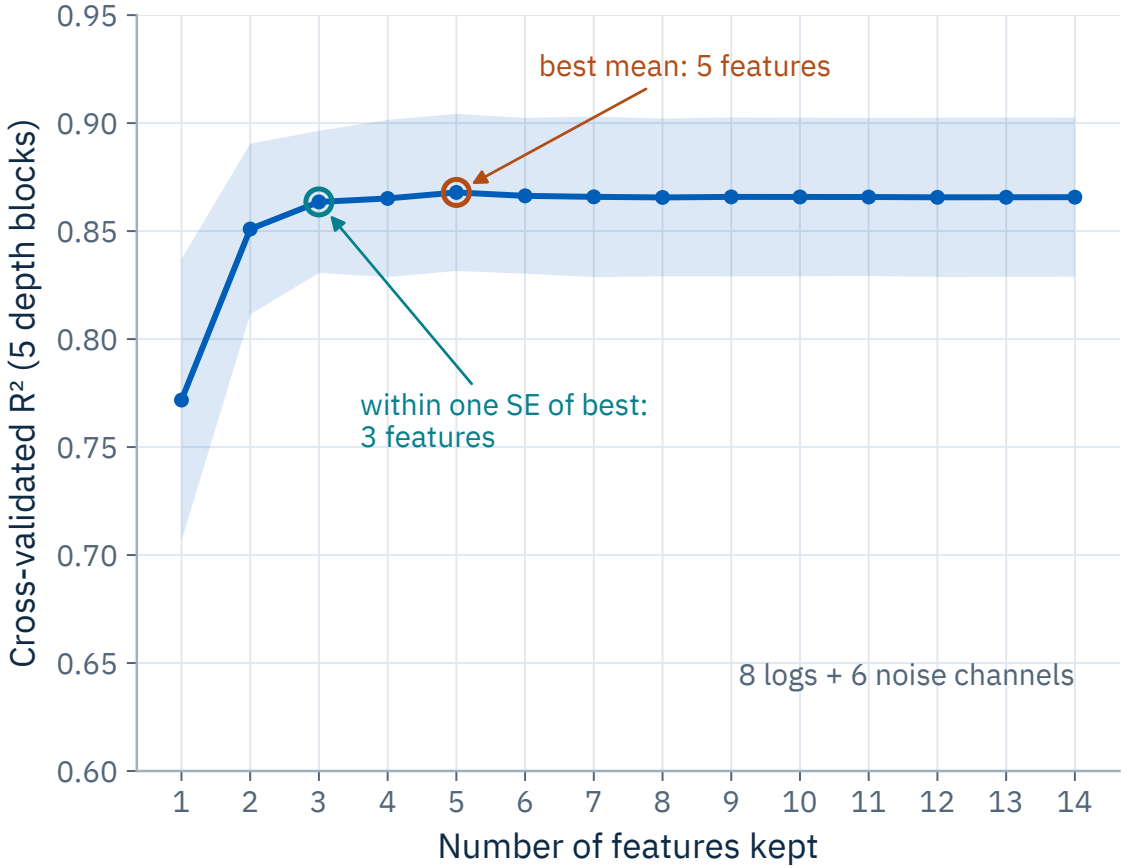
A backward search that uses the model's own ranking in every round.

1. **Fit** the model on all current features
2. **Rank** features by coefficient size or importance
3. **Drop** the weakest one (step = 1)
4. **Repeat** until one feature remains
5. **Choose** the size with the best CV score

◆ RFECV repeats the whole loop inside every training fold, so each size is scored on unseen rows.

Cross-validation chooses how many features to keep

- R^2 rises from 0.77 with one log to 0.87 with five, then flattens
- Six noise channels neither help nor clearly hurt this linear model
- Within one standard error of the best: three features
- Folds are contiguous depth blocks, not shuffled rows



RFECV with linear regression; synthetic log suite plus six noise channels; band = ±1 SD across five folds.

Wrapper selectors nest inside an outer cross-validation

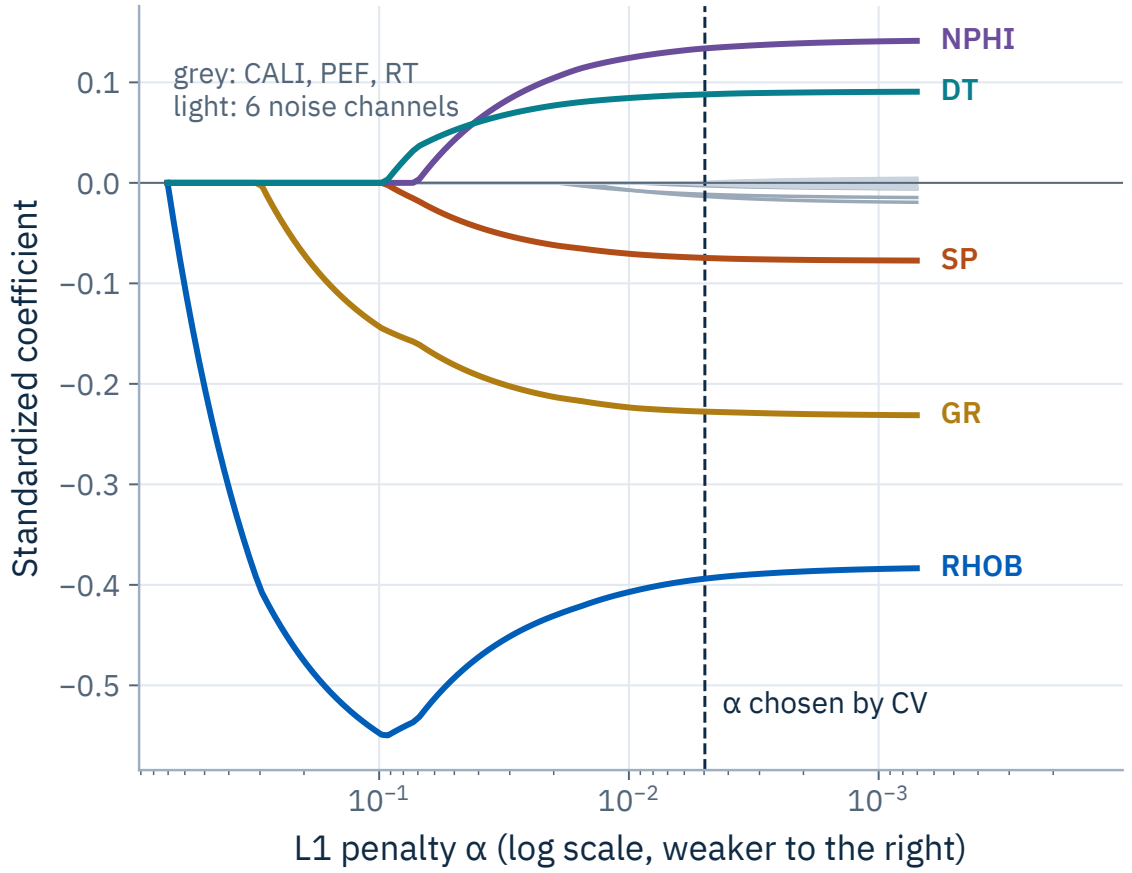
```
from sklearn.feature_selection import RFECV, SequentialFeatureSelector
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import KFold, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

blocks = KFold(n_splits=5)            # contiguous depth blocks, no shuffle
rfe = RFECV(LinearRegression(), cv=blocks, scoring="r2")
sfs = SequentialFeatureSelector(LinearRegression(), direction="forward",
                                n_features_to_select="auto", tol=0.005,
                                cv=blocks, scoring="r2")
for selector in (rfe, sfs):
    model = make_pipeline(StandardScaler(), selector, LinearRegression())
    outer = cross_val_score(model, X, y, cv=blocks, scoring="r2")
    print(type(selector).__name__, outer.mean().round(3))
```

The inner search runs on each outer training fold only; the outer score is the honest one.

L1 penalties drive weak coefficients exactly to zero

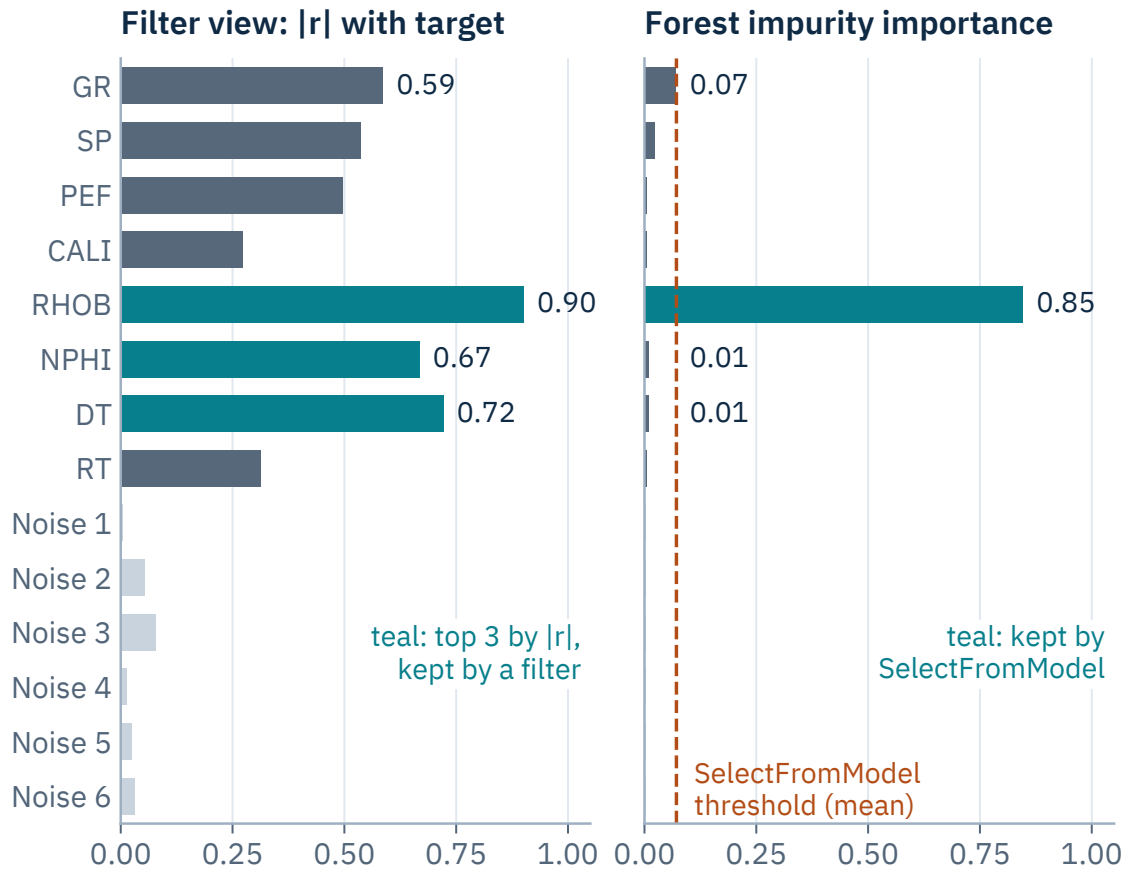
- As α shrinks, features enter: RHOB, then GR, then SP, DT and NPHI
- RHOB's coefficient shrinks once correlated NPHI and DT join
- Standardize first; pick α by CV (dashed line); see Module 1 on the lasso



Lasso path on the synthetic log suite plus six noise channels (target: log permeability).

A forest gives redundant logs almost no importance

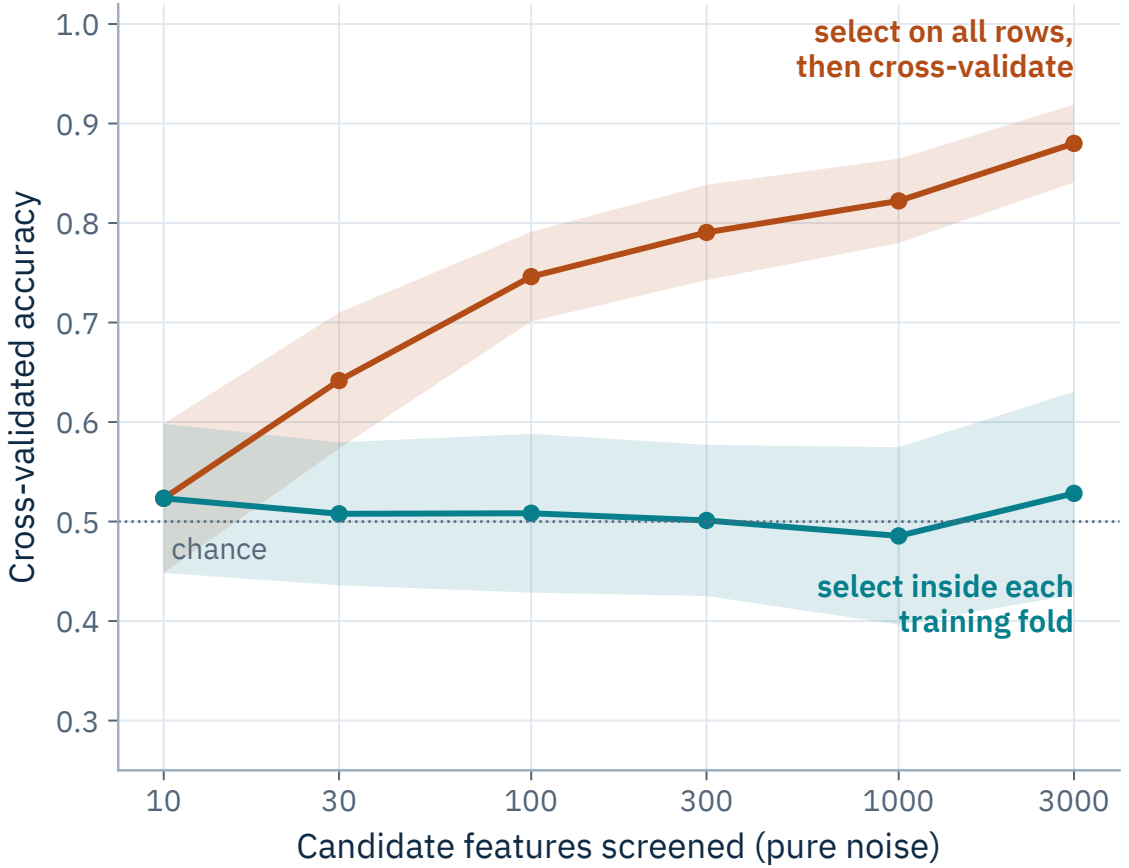
- A filter rates NPHI and DT highly: $|r| = 0.67$ and 0.72 with the target
- The forest credits RHOB with 0.85; NPHI and DT get 0.01, near noise
- SelectFromModel at the mean importance keeps only RHOB
- Redundant is not useless: without RHOB, NPHI and DT carry the signal



Random forest (300 trees, seed 0) on the synthetic log suite plus six noise channels; target log permeability.

Selection must happen inside cross-validation

- Pure noise: 60 rows, random labels, keep the 10 best-scoring features
- Selected on all rows: accuracy climbs from 0.52 to 0.88 with more candidates
- Selected inside each training fold: chance at every size



30 synthetic pure-noise datasets per point (seeds 2000–2029); SelectKBest (k = 10) and logistic regression; 5-fold CV; bands = ±1 SD.

⚠ Any step that looks at the target belongs inside the fold, as in Module 2's leakage demonstration.

The selector belongs inside the pipeline

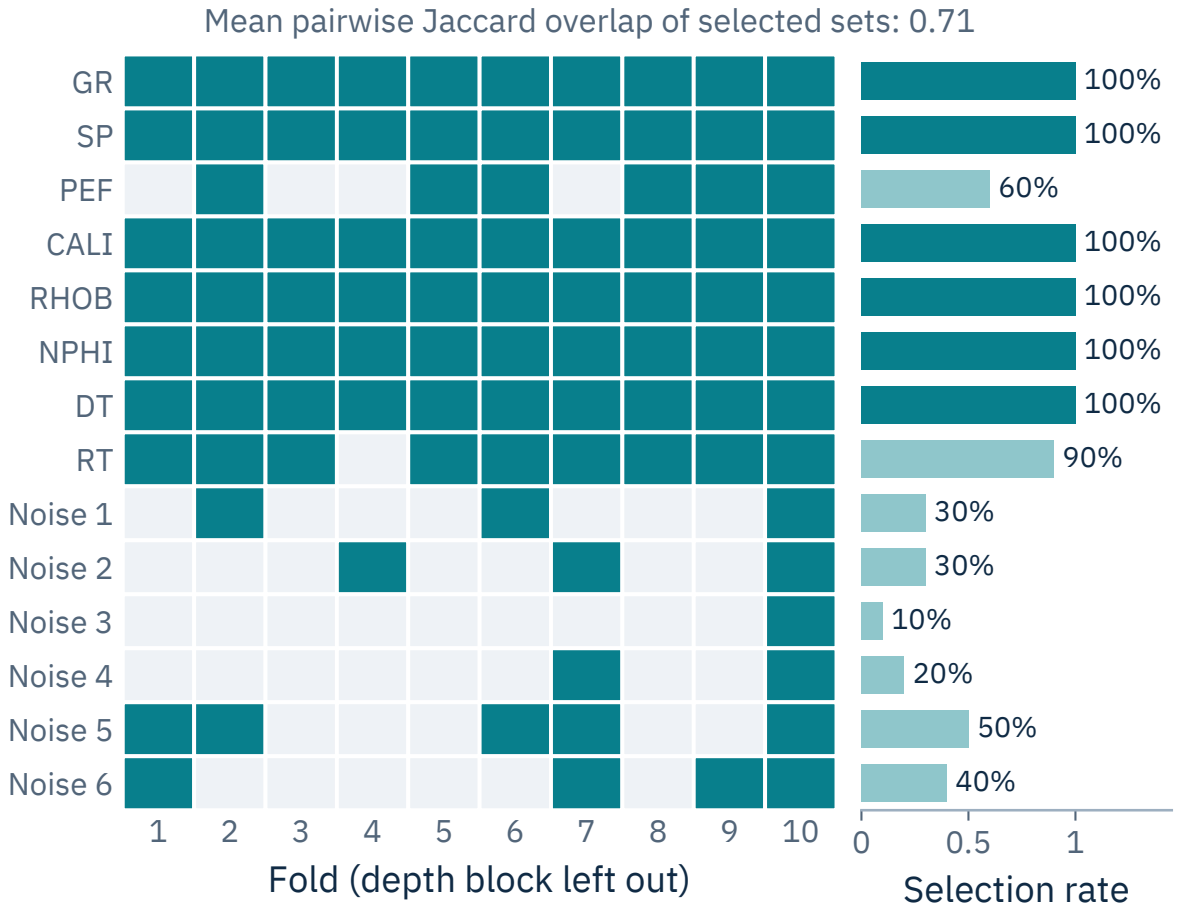
```
from functools import partial
from sklearn.feature_selection import SelectKBest, mutual_info_classif
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GroupKFold, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

mi = partial(mutual_info_classif, random_state=0)
model = make_pipeline(StandardScaler(), SelectKBest(mi, k=10),
                      LogisticRegression(max_iter=1000))
scores = cross_val_score(model, X, y, groups=well_id,
                         cv=GroupKFold(n_splits=5), scoring="f1_macro")
print(scores.mean().round(3), scores.std().round(3))
```

Each fold re-ranks the features on its own training wells; k can be tuned in the same grid search.

Selected sets change from fold to fold

- LassoCV refitted in ten depth-block folds
- Six logs chosen in every fold; noise channels in 10–50% of folds
- Mean Jaccard overlap between fold selections: 0.71
- Report selection frequencies, not one "final" list



Synthetic log suite plus six noise channels (seeds 11, 5).

Choose a selection family by cost and blind spot

Family	Cost	Sees interactions	Main risk
Filter	Low	No	Redundant picks; misses joint effects
Wrapper	High	Through the model	Overfitting the search
Embedded	Moderate	Model-specific	Arbitrary picks among correlated features

All three need the same honest validation as the final model.

◆ A robust default: drop dead and duplicate channels first, then an embedded or wrapper method inside grouped CV.

PRACTICE

6 min · Pairs

Find the leak in this selection workflow

Thirty wells, 400 candidate drilling features, target ROP. An analyst ranks features by $|r|$ with ROP on all rows, keeps the top 25, then reports a GroupKFold score by well.

1. Where does information leak, and why does grouping by well not prevent it?
2. Restructure the workflow so that the score is honest.
3. What should the report say about the 25 selected features?

DISCUSSION

A leak-free selection workflow has four steps

The order is the point: the split comes first, and everything that learns sits inside the fold.

1. **Split**
   by well, before anything is learned
2. **Inside each training fold**
   scale, rank, select, fit
3. **Score**
   on held-out wells only
4. **Report**
   score spread and selection frequency

- Tune k in an inner loop, and keep final test wells untouched until the end.

PART 3 OF 6

03

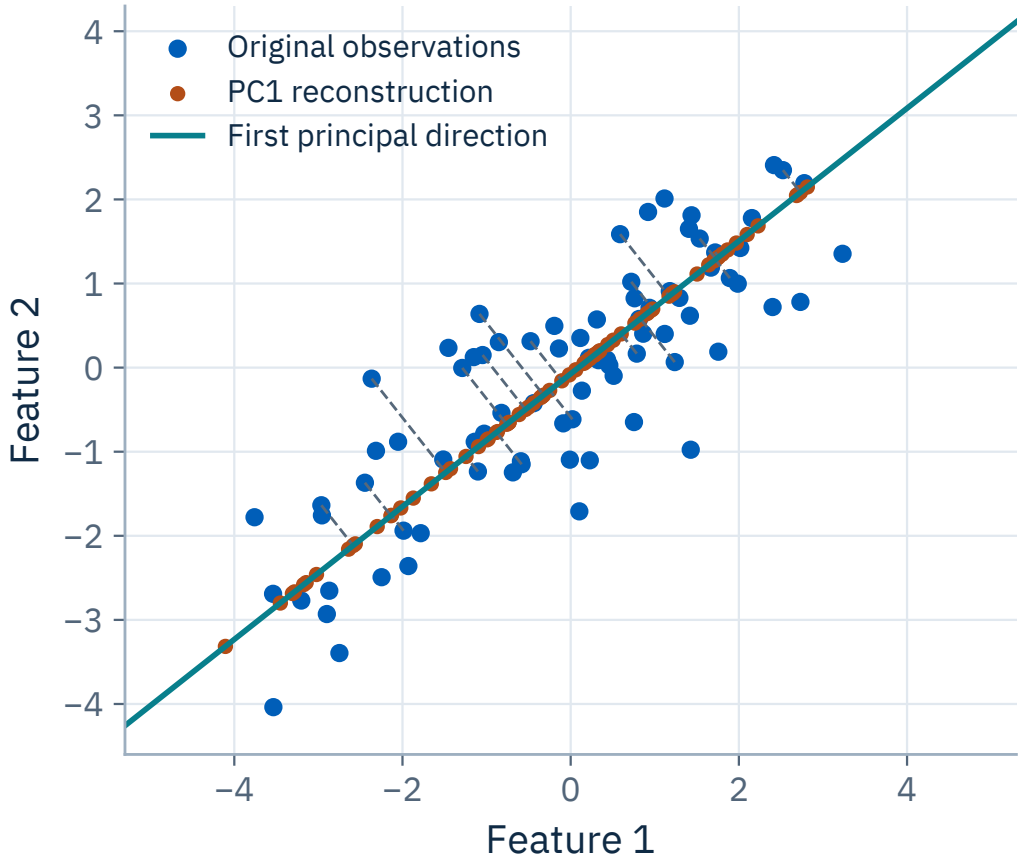
Principal components in depth

Orthogonal directions summarize variance in centred data.

- Covariance, eigenvectors and the SVD
- Scaling choices and the number of components
- Loadings, biplots and reconstruction error
- Whitening, batches and pipelines

The first principal direction keeps the most spread

- PC1 is the line along which projected points vary most
- Equivalently, it minimizes squared perpendicular distances (dashed)
- Components are orthogonal, ordered by variance, and sign-arbitrary

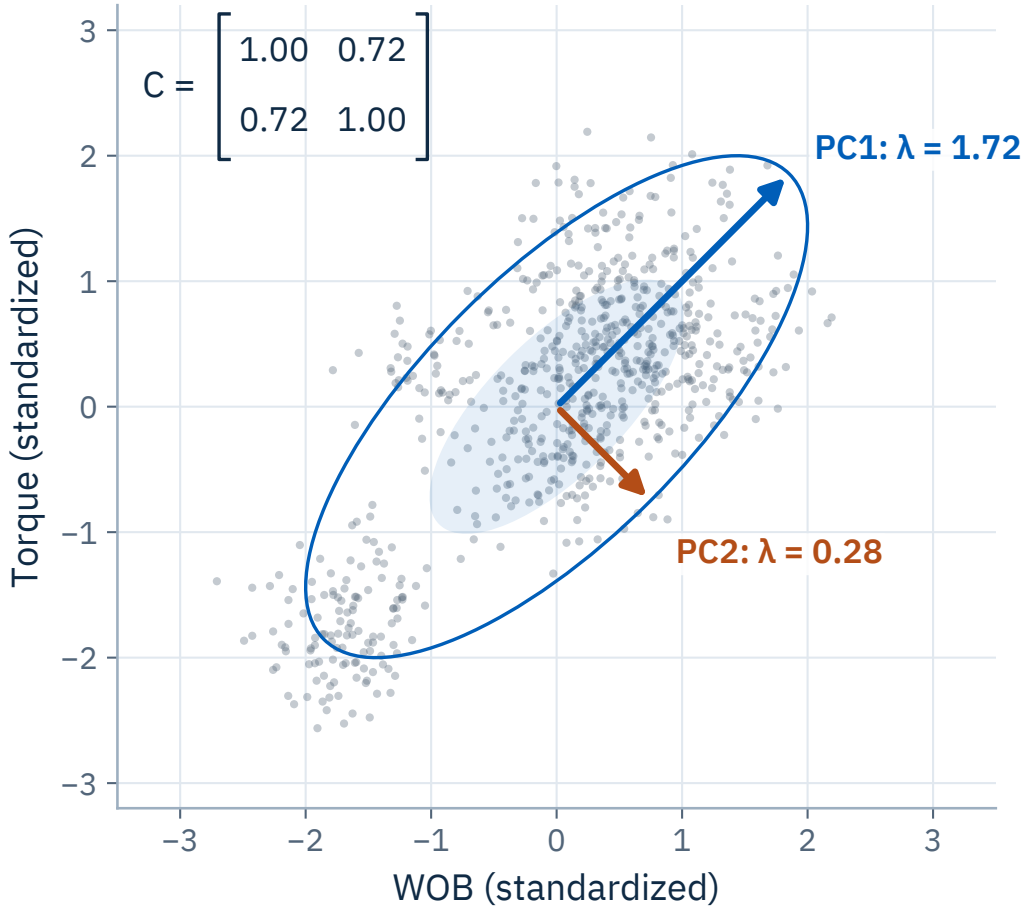


PCA computed from a synthetic correlated two-feature dataset. Dashed segments show reconstruction residuals after projection onto PC1.

The covariance matrix sets the principal axes

$$C v_j = \lambda_j v_j$$

- C: covariance of the centred data, d × d
- Eigenvectors v: the principal directions
- Eigenvalues λ: the variance along each direction
- Standardized WOB and torque: r = 0.72, so λ = 1 ± r = 1.72 and 0.28

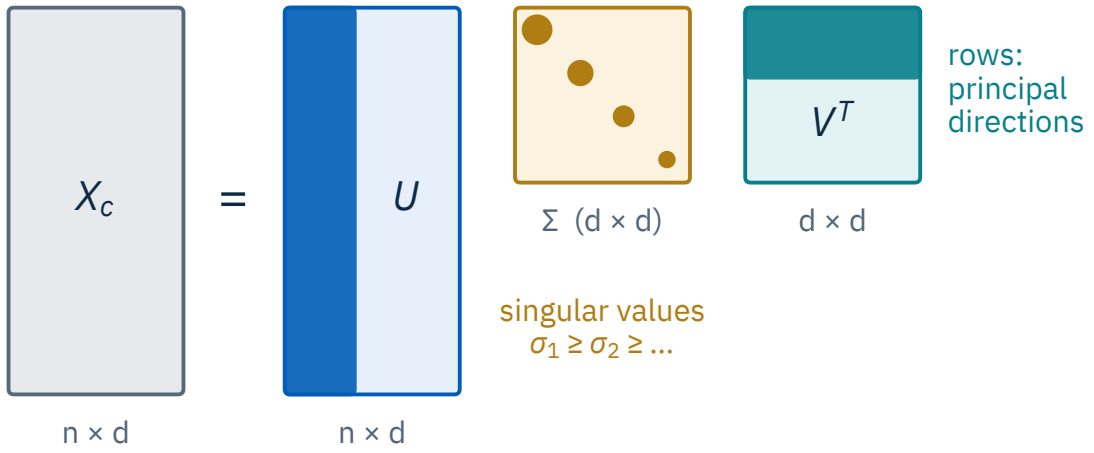


Synthetic drilling channels (seed 4); arrows have length 2√λ.

The SVD delivers directions, variances and scores

$$X_c = U\Sigma V^T, \qquad Z = X_c V_k$$

- Centred data: n observations × d features
- Rows of V^T are the principal directions
- Keeping k directions gives the n × k score matrix Z



Keep the first k components (shaded):



$$Z = X_c V_k = U_k \Sigma_k$$

scores; variance of score j: $\lambda_j = \sigma_j^2/(n-1)$

Schematic of the thin singular value decomposition.

Centring and scaling answer different questions

Standardize: correlation-matrix PCA

- Each feature contributes in unitless terms
- Default for mixed units: psi, rpm, klbf, ft/h

VS

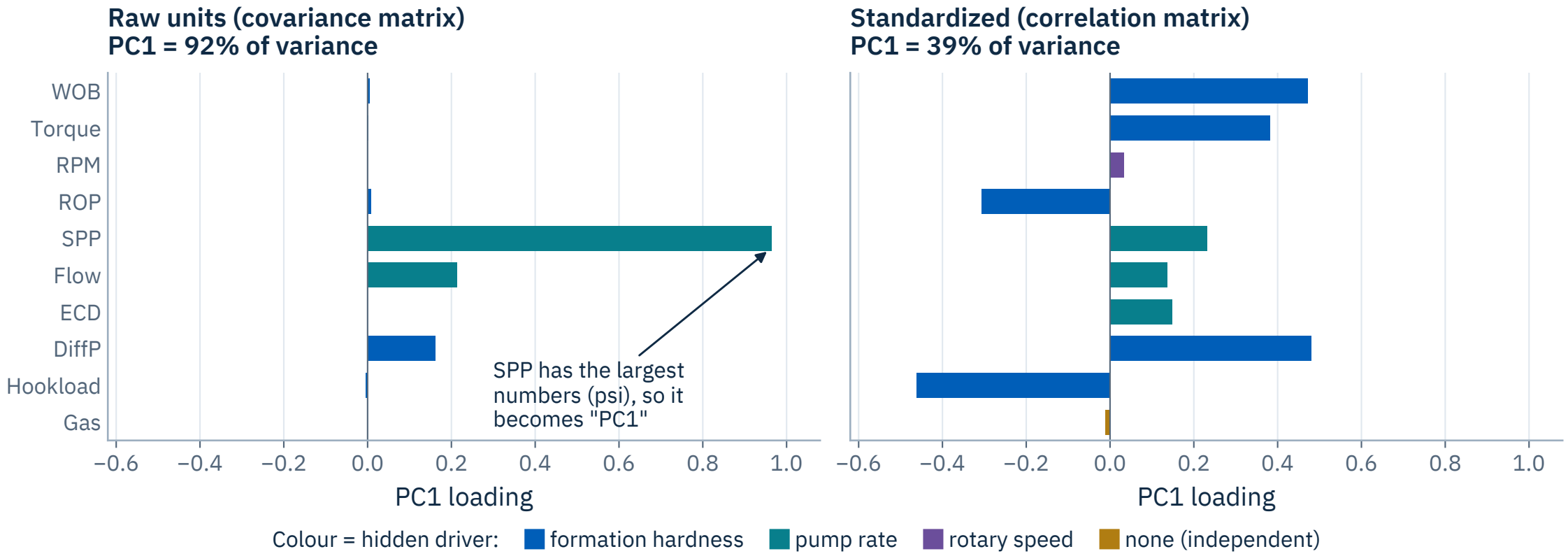
Keep units: covariance-matrix PCA

- Absolute variation is the point
- Only when units are comparable, such as one sensor type

◆ Always centre, and fit means and scales on training data only.

Standardization changes the answer

Same ten channels, same algorithm: only the scaling differs.



Synthetic drilling channels in engineering units (seed 4). Bar colours mark the driver behind each channel.

PRACTICE

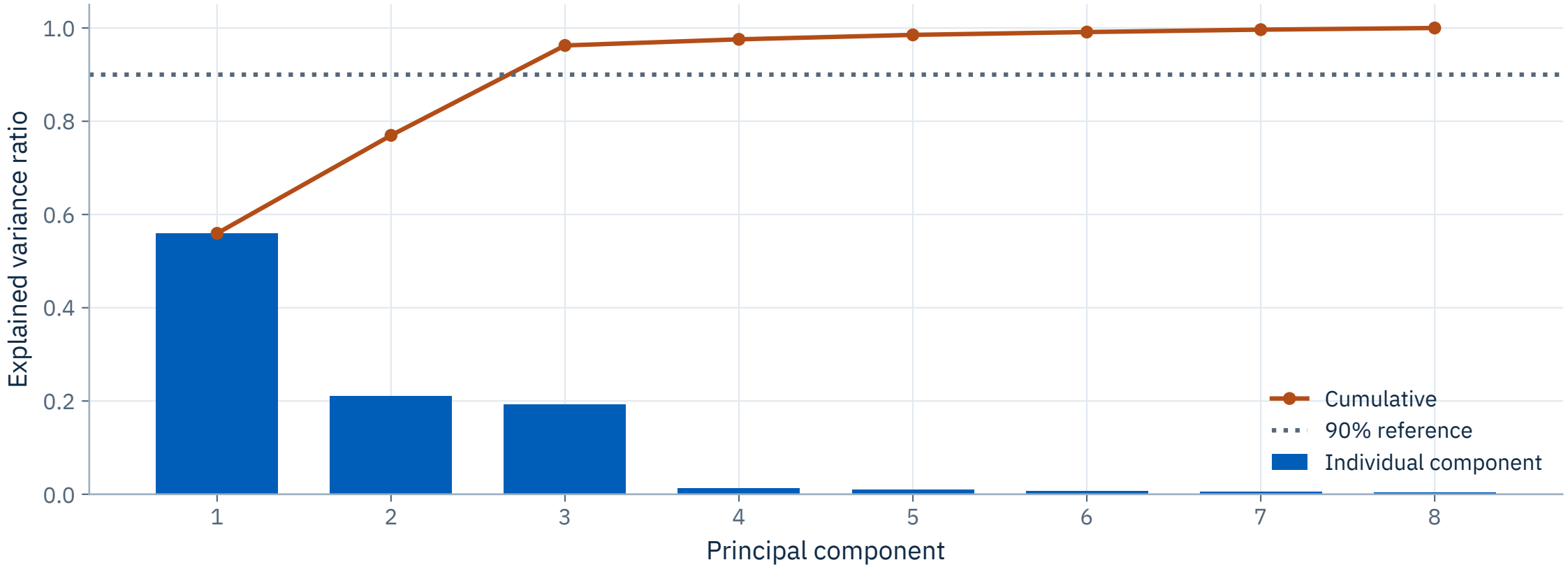
5 min · Individual

A projection by hand shows what PCA discards

A centred observation is $x = [2, 0]$. The first direction is $v = [1, 1] / \sqrt{2}$.

1. Calculate its score on this direction.
2. Reconstruct the observation using this single component.
3. Calculate the squared reconstruction error.
4. What share of the point's squared length did PC1 keep?

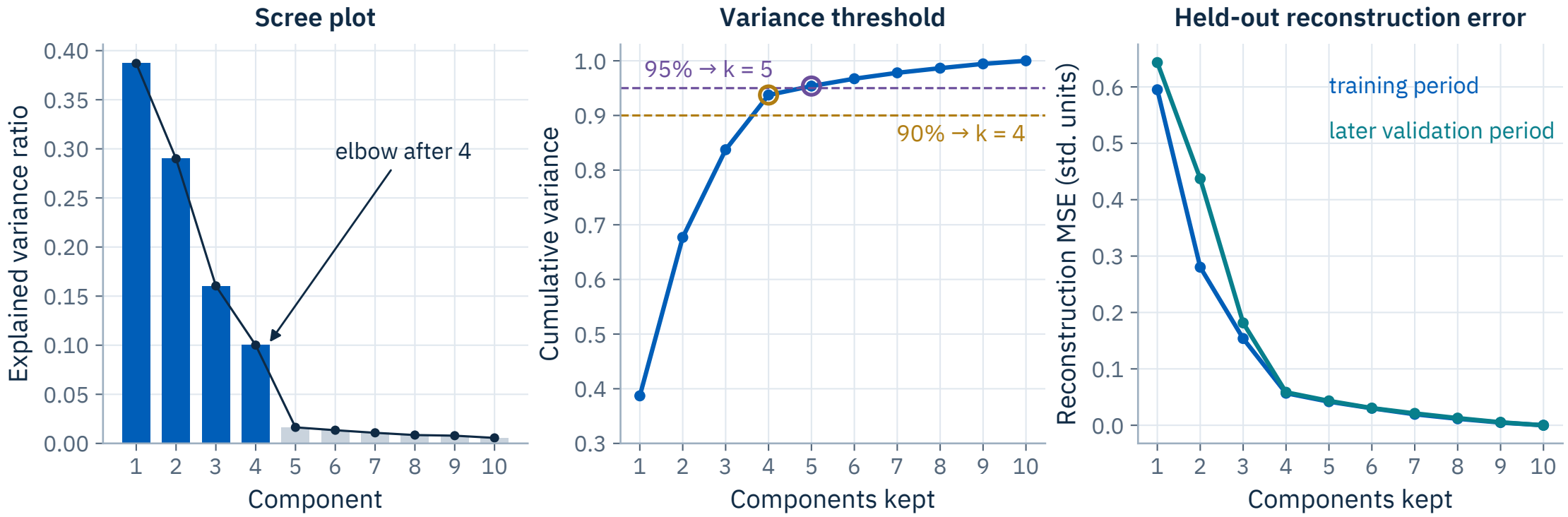
Explained variance describes retained spread



PCA on a generated correlated feature matrix. Bars show individual variance ratios; the line shows their cumulative sum.

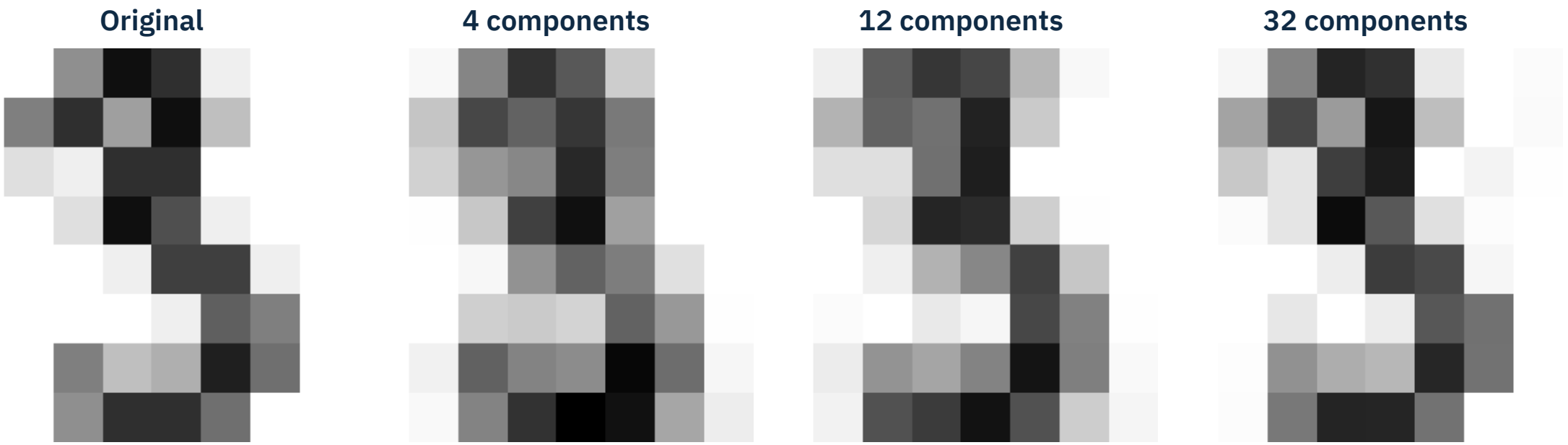
Three rules for k give related, not identical, answers

Scree elbow after 4; 90% of variance needs 4 components, 95% needs 5; held-out error flattens after 4.



Synthetic drilling channels (seed 4). Left and middle: all 720 minutes. Right: PCA fitted on minutes 0–429, validated on minutes 430–719.

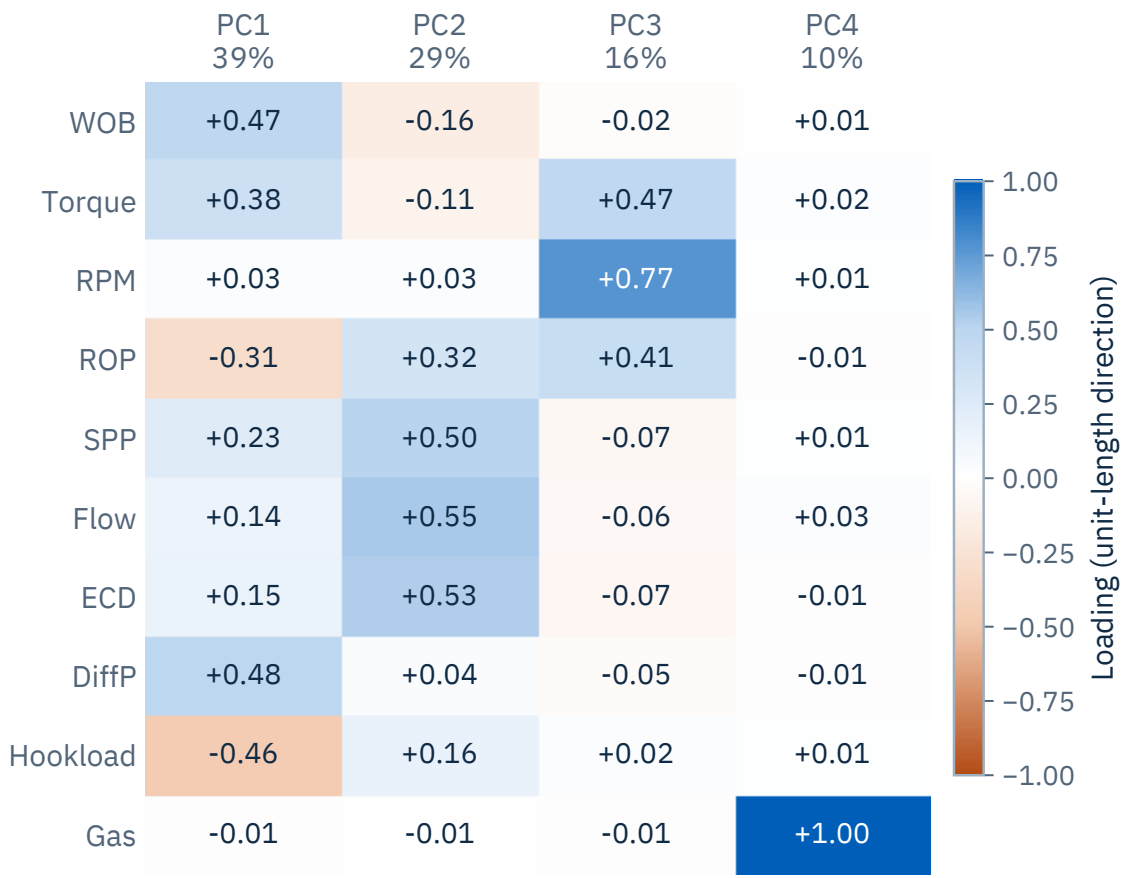
Reconstruction makes information loss visible



PCA fitted to the bundled scikit-learn digits (8 × 8 images). Linear compression, not a neural autoencoder.

Loadings connect components to features

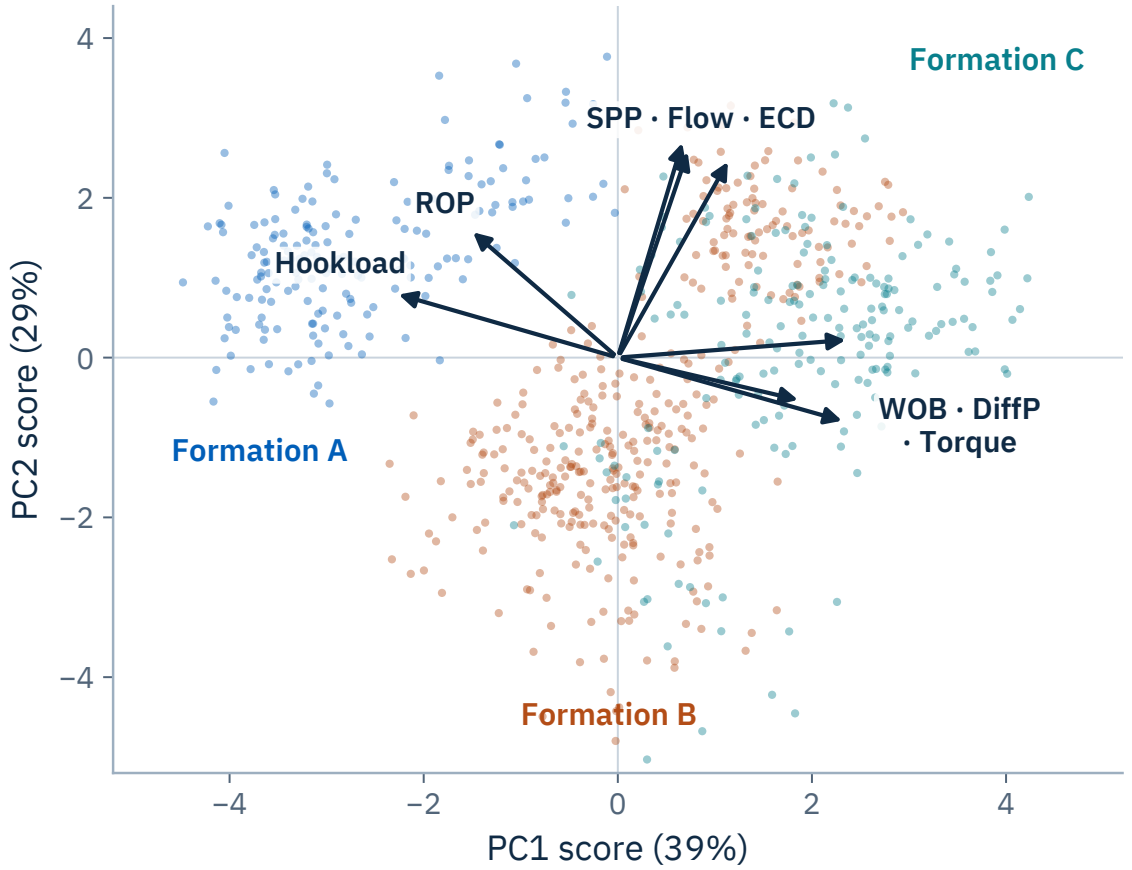
- Large |*loading*|: the feature drives that direction
- Opposite signs mark a contrast: WOB up, hook load down on PC1
- A component's overall sign can flip without changing the fit
- PC4 is gas alone: a whole component for one channel



Synthetic drilling channels (seed 4), standardized. Rows of components_, signs fixed for display.

A biplot shows observations and loadings together

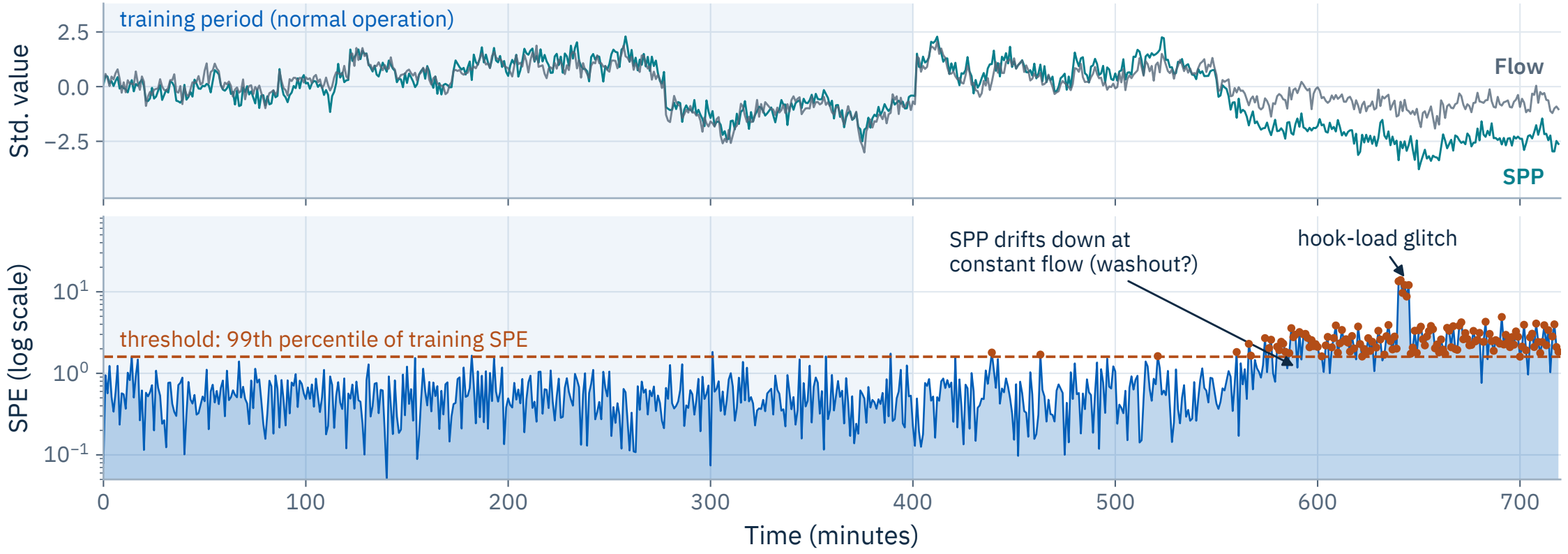
- Points: PC1–PC2 scores, coloured by formation after fitting
- Arrows: loadings, rescaled for display
- Formation A sits toward high hook load and ROP, low WOB
- Angles approximate correlations only if two PCs dominate (68% here)



Synthetic drilling channels (seed 4); arrows are loadings × 5 for display. RPM and gas load on PC3–PC4.

Reconstruction error flags unusual observations

Fit PCA on normal operation; score each new minute by its squared prediction error (SPE).



Synthetic drilling channels (seed 4) with an injected SPP drift and a hook-load glitch; PCA with k = 4 fitted on minutes 0–399.

Whitening rescales every component to unit variance

$$z_j = \frac{x_c^T v_j}{\sqrt{\lambda_j}}$$

- `PCA(whiten=True)` divides each score by the square root of its eigenvalue
- Whitened scores are uncorrelated and have equal variance
- A standard first step for ICA and some distance-based models
- It inflates noise in small components, so keep k modest

Incremental PCA fits data in batches

```python
import pandas as pd
from sklearn.decomposition import IncrementalPCA
from sklearn.preprocessing import StandardScaler

cols = ["WOB", "TORQUE", "RPM", "ROP", "SPP", "FLOW"]
scaler = StandardScaler()
for chunk in pd.read_csv("rig_1hz.csv", usecols=cols, chunksize=100_000):
    scaler.partial_fit(chunk)                    # pass 1: means and SDs

ipca = IncrementalPCA(n_components=4)
for chunk in pd.read_csv("rig_1hz.csv", usecols=cols, chunksize=100_000):
    ipca.partial_fit(scaler.transform(chunk))    # pass 2: components
print(ipca.explained_variance_ratio_.round(3))
```

Useful when a year of 1 Hz rig data does not fit in memory; check against batch PCA on a sample.

PCA belongs inside the prediction pipeline

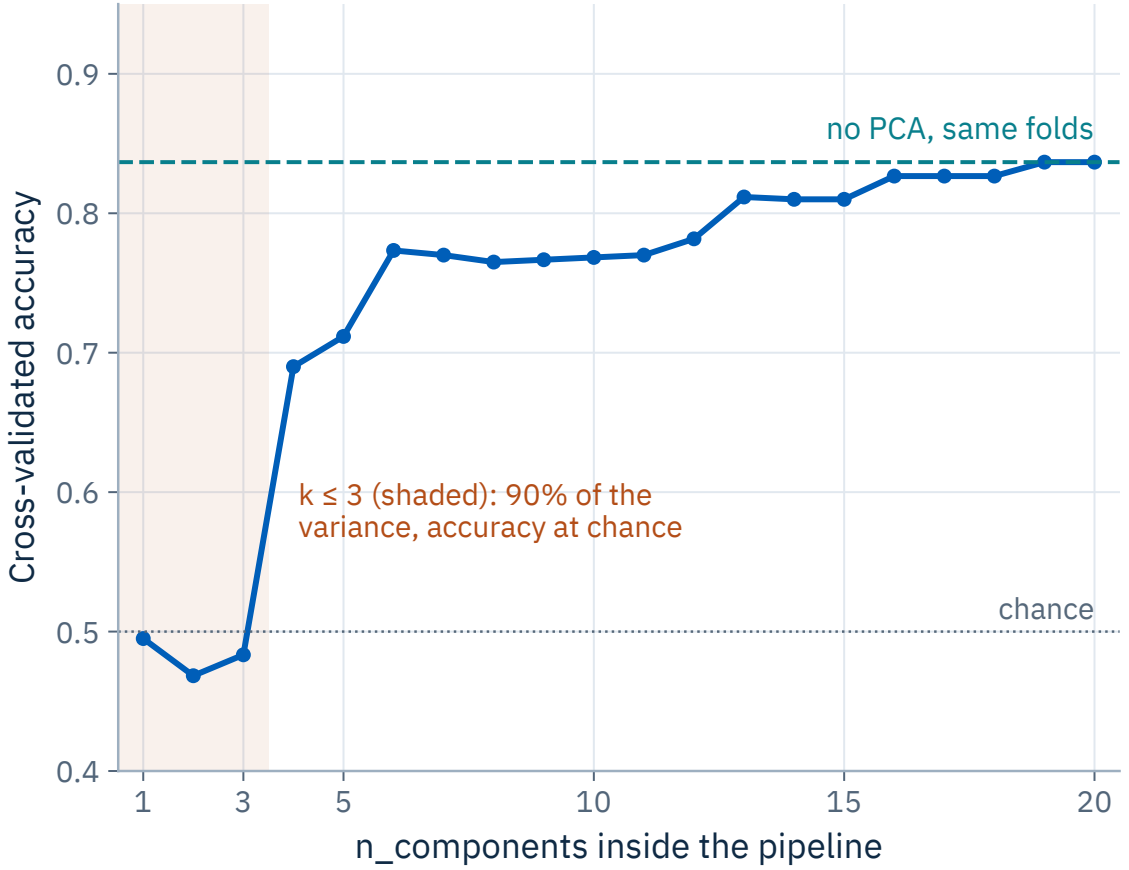
```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV, GroupKFold

model = make_pipeline(StandardScaler(), PCA(),
                      LogisticRegression(max_iter=1000))
grid = {"pca__n_components": [2, 4, 6, 8, 10]}
search = GridSearchCV(model, grid, cv=GroupKFold(n_splits=5),
                      scoring="f1_macro")
search.fit(X_train, y_train, groups=wells_train)
print(search.best_params_, round(search.best_score_, 3))
```

Compare with the same pipeline without PCA, on the same folds.

Tune the number of components against the task

- Three nuisance factors hold 90% of the variance and no class signal
- With k ≤ 3 the classifier sits at chance (0.47–0.50)
- It matches the no-PCA baseline (0.84) only at k = 19



Synthetic 20-feature data (seed 3); pipeline: StandardScaler, PCA, logistic regression; 5-fold CV.

PRACTICE

5 min Pairs

PCA limitations are testable

Pressure explains most variance, but a small vibration feature predicts failures.

1. Would keeping only high-variance components necessarily help classification?
2. What experiment would test this?
3. Which method from Part 2 could keep the vibration feature?

LABORATORY

60 min

Lab 1: PCA on drilling data

- Check the correlation matrix first; treat Depth as an index, not a feature
- Standardize ROP, RPM, Torque and WOB; fit PCA with all components
- Choose k from scree and cumulative plots, and justify the threshold
- Read loadings and the biplot; report reconstruction error in original units
- Extension: repeat PCA on raw units and compare PC1 with the standardized fit

NOTEBOOK

`exercises/01_pca/PCA_Drilling_Student.ipynb`

EVIDENCE TO SUBMIT

- Scree and cumulative-variance plots with your chosen k
- A loadings table with one sentence per component
- Reconstruction RMSE per feature in original units

DISCUSSION

Lab 1 debrief: this drilling table barely compresses

The course file drilling.csv, analysed as in the notebook:

28.5%

PC1 share after standardizing

4 of 4

components needed for 90% of variance

89%

PC1 share in raw units: WOB alone

- A flat spectrum is a finding: the four channels are nearly uncorrelated ($|r| \leq 0.13$), so there is little to compress.

PART 4 OF 6

04

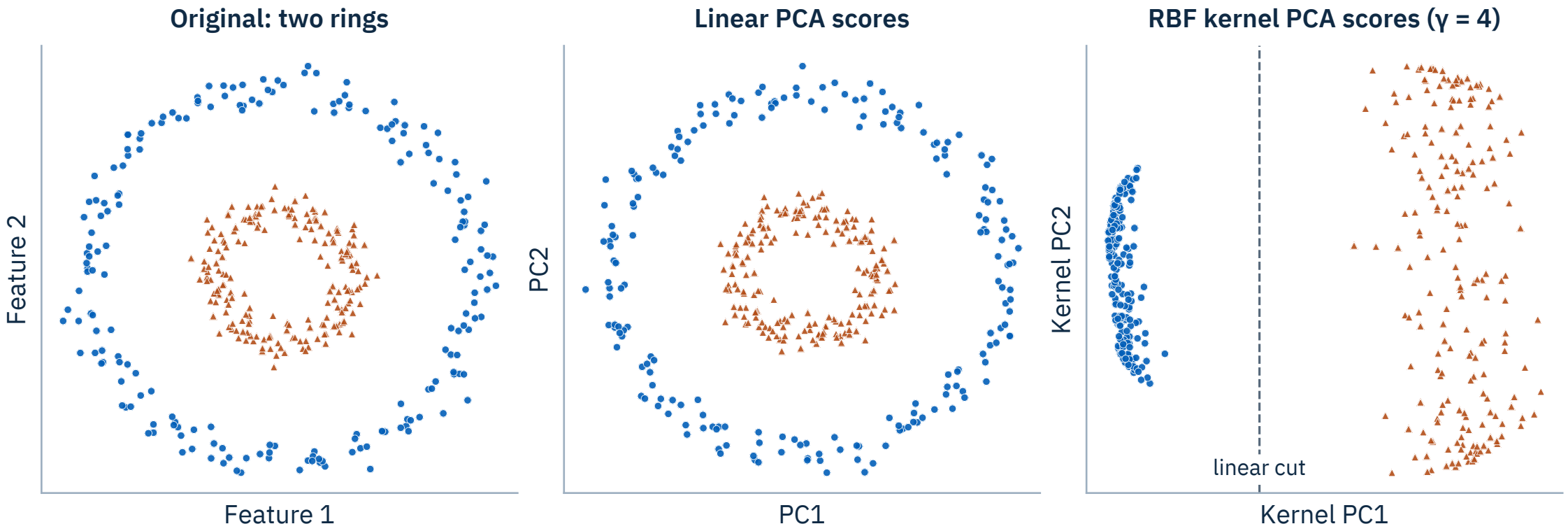
Beyond linear PCA

Change the geometry or the objective of PCA.

- Kernel PCA for curved structure
- LDA: a supervised projection
- ICA and NMF: independent sources, additive parts
- Autoencoders: a preview of the neural-network course

Kernel PCA can unfold what linear PCA cannot

PCA runs on an n × n similarity (kernel) matrix instead of the d × d covariance matrix.



scikit-learn make_circles (400 points, seed 0); RBF kernel, γ = 4. Colours show ring membership, which is never used in fitting.

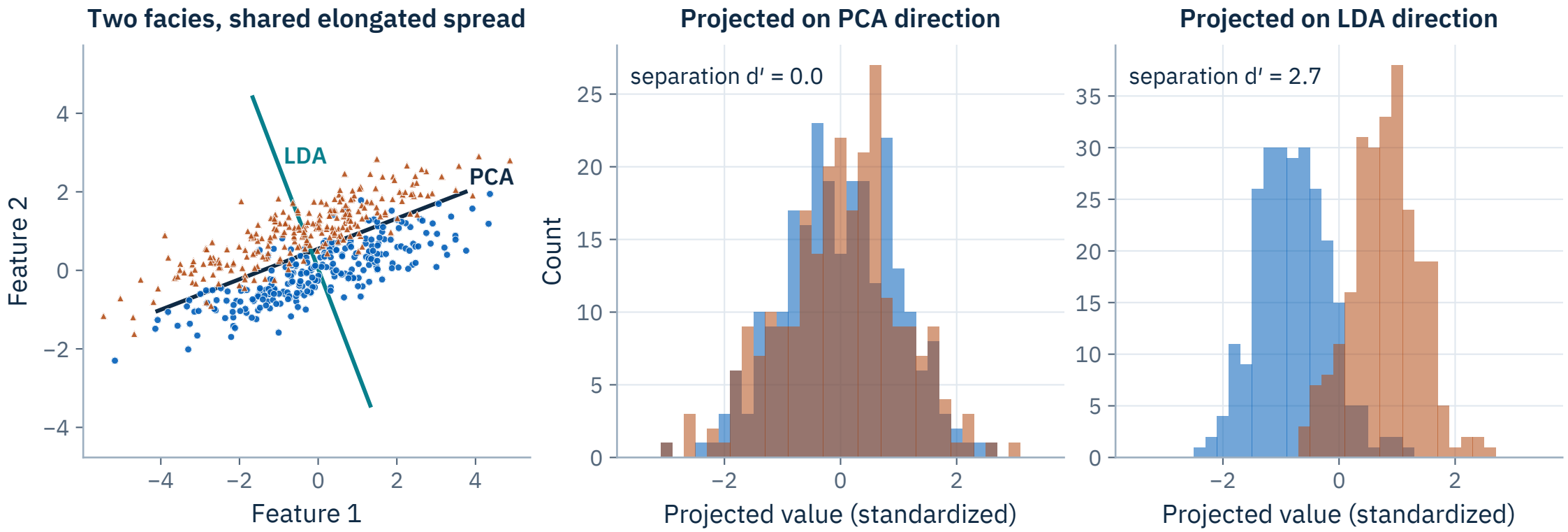
Kernel PCA trades interpretability for flexibility

	Linear PCA	Kernel PCA
Works on	d × d covariance	n × n kernel matrix
Captures	Linear directions	Curved structure, via the kernel
Memory and time	Modest for large n	Grows with n^2 and n^3
Interpretation	Loadings per feature	No loadings; approximate pre-images
New observations	Exact projection	`transform()` via kernels to training rows

◆ Tune γ against a downstream score; without one, prefer linear PCA.

LDA projects to separate labelled classes

PCA follows the largest spread; LDA follows the direction that separates the labelled classes.



Synthetic two-class data with a shared covariance (seed 2). d′ = difference in class means / pooled standard deviation.

PCA and LDA answer different questions

PCA

- Unsupervised: never sees labels
- Maximizes total projected variance
- Up to d components

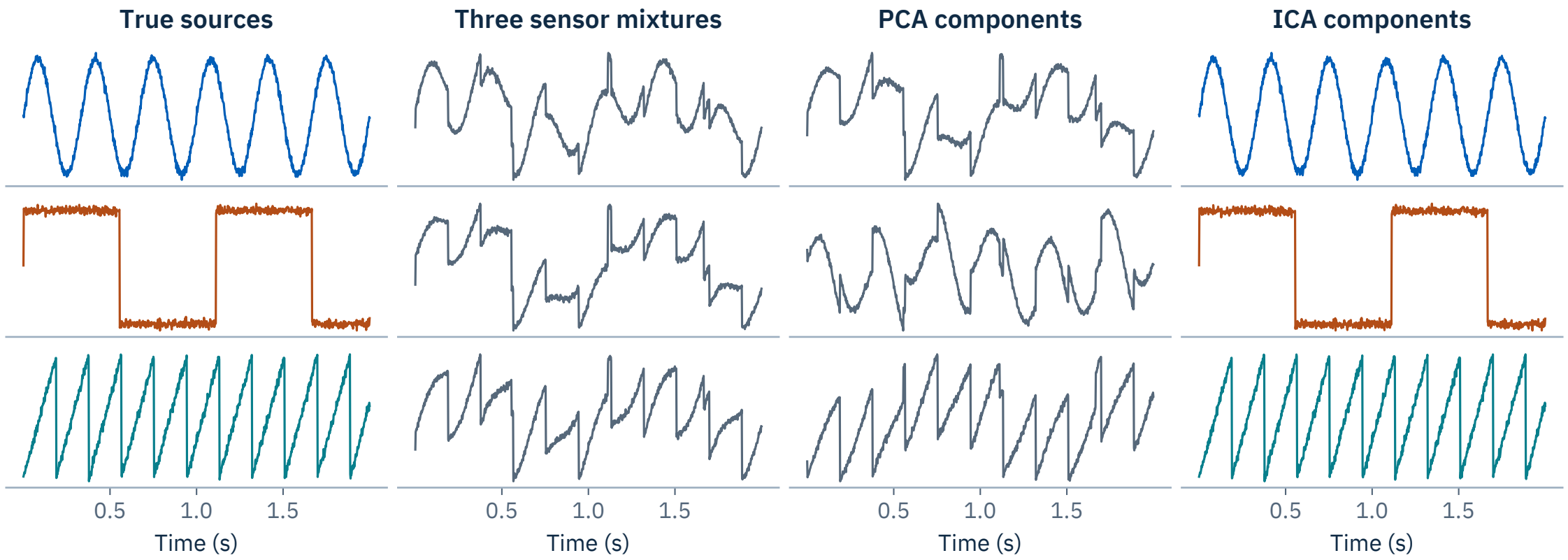
vs

LDA

- Supervised: uses class labels
- Maximizes between- over within-class variance
- At most C – 1 components for C classes

◆ LDA learns from labels, so fit it inside the training folds, like any model.

ICA unmixes sources that PCA only decorrelates



Synthetic sources mixed by a fixed 3 × 3 matrix; FastICA (seed 0). ICA rows are reordered and sign-matched for display, because ICA cannot recover order, sign or scale.

ICA works only when its assumptions hold

Assumptions

- Statistically independent sources
- At most one Gaussian source
- Linear, instantaneous mixing; sensors ≥ sources

Ambiguities

- Order, sign and scale are arbitrary
- Match components to physics after fitting

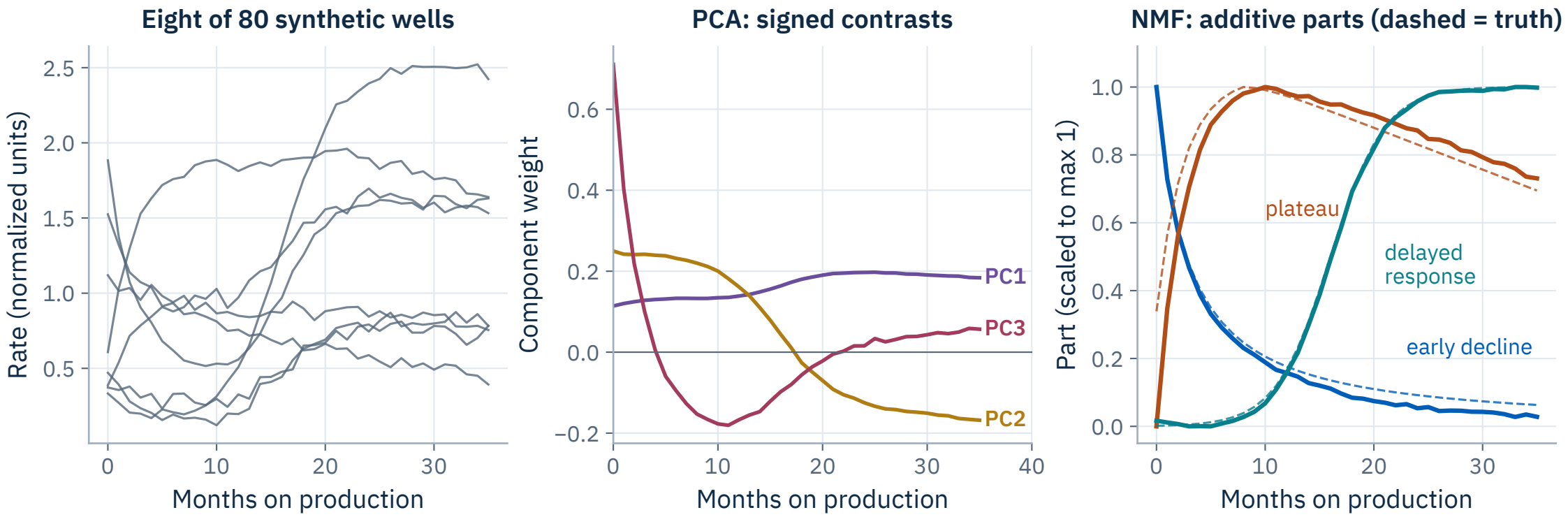
Energy uses

- Separating overlapping vibration or acoustic sources
- Removing an interference shared by many sensors

◆ FastICA whitens the data first: PCA is its first step.

NMF builds profiles from additive parts

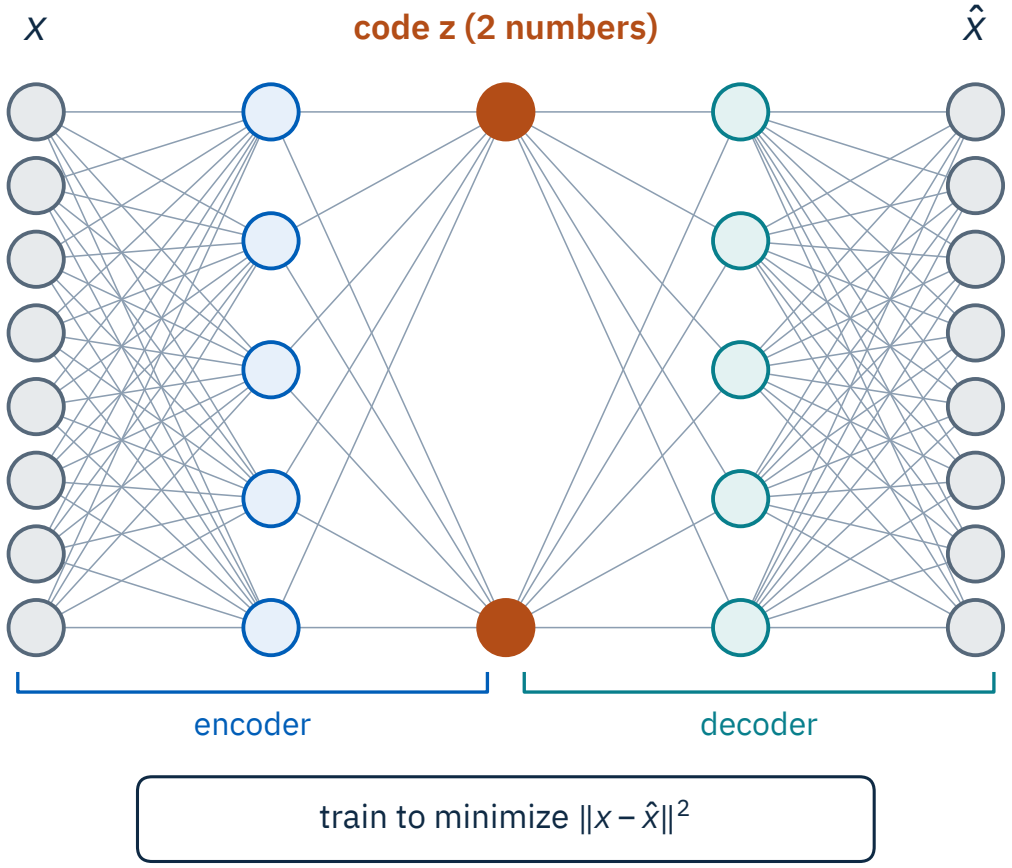
Non-negative data (rates, spectra, concentrations) give non-negative parts and weights.



Synthetic production profiles (seed 9); NMF with k = 3 (seed 0); PCA with k = 3 shown for contrast.

Autoencoders learn a nonlinear bottleneck

- Encoder: $x \rightarrow$ short code z; decoder: $z \rightarrow \hat{x}$
- Linear layers with squared error span the same subspace as PCA
- Nonlinear layers can follow curved structure, at a cost in data and care
- Reconstruction error again serves as an anomaly score



Schematic. Eight inputs, a two-number code and eight outputs.

PRACTICE

5 min · Pairs

Each goal points to a different method

Name one method for each goal, and one check you would run:

1. Summarize 20 correlated rig channels for storage within an error budget
2. A 2-D view that best separates three labelled facies for a report
3. Unmix pump and motor vibration recorded by three accelerometers
4. Describe non-negative production profiles as a few typical shapes
5. One unsupervised score that separates two concentric groups in a cross-plot

PART 5 OF 6

05

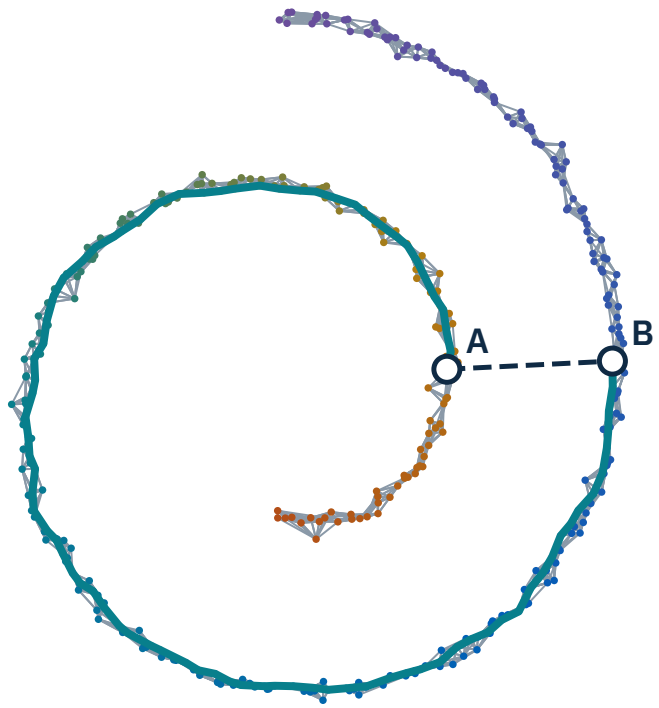
Manifold learning

An embedding is a view of relationships, not a physical map.

- Geodesic distances: Isomap and LLE
- t-SNE: neighbour probabilities
- UMAP: neighbourhood graphs
- Trustworthiness, seeds and batch effects

Curved data need distances along the surface

- A and B are 6.2 apart in a straight line, 60.3 along the data
- A k-nearest-neighbour graph approximates the surface
- Shortest paths on the graph estimate geodesic distance
- Isomap then embeds these distances with classical MDS



grey lines: graph of 7 nearest neighbours

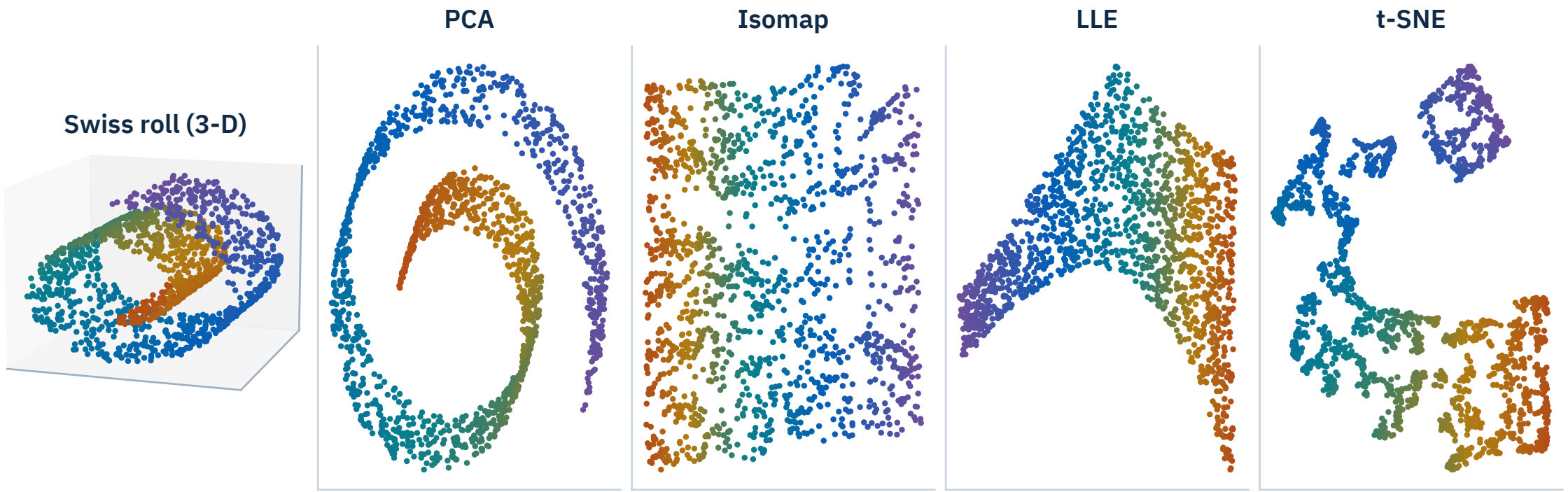
straight line A–B: 6.2

path along the graph: 60.3

Synthetic noisy spiral (seed 0, 400 points); 7-nearest-neighbour graph; scipy shortest path. Colour = position along the spiral.

Methods unroll the Swiss roll differently

Colour shows position along the roll; it is never used in fitting.



scikit-learn make_swiss_roll (1,500 points, noise 0.05, seed 0); Isomap k = 10, LLE k = 12, t-SNE perplexity 30.

Isomap and LLE keep different local facts

Isomap

- Neighbour graph, then geodesic distances
- Classical MDS on those distances
- Keeps global shape; one short-circuit edge can fold it

vs

LLE

- Rebuild each point from its neighbours
- Keep those weights in the 2-D map
- Keeps local geometry; global shape can distort

◆ Both hinge on n_neighbors: too small breaks the graph, too large short-circuits the roll.

t-SNE turns distances into neighbour probabilities

Four ideas explain every t-SNE map, and what it cannot show.

1

Similarities

Gaussian neighbourhoods in the input space

2

Heavy tail

Student-t similarities in the map

3

Optimize

move points until the two agree

4

Check

perplexity, initialization and seed

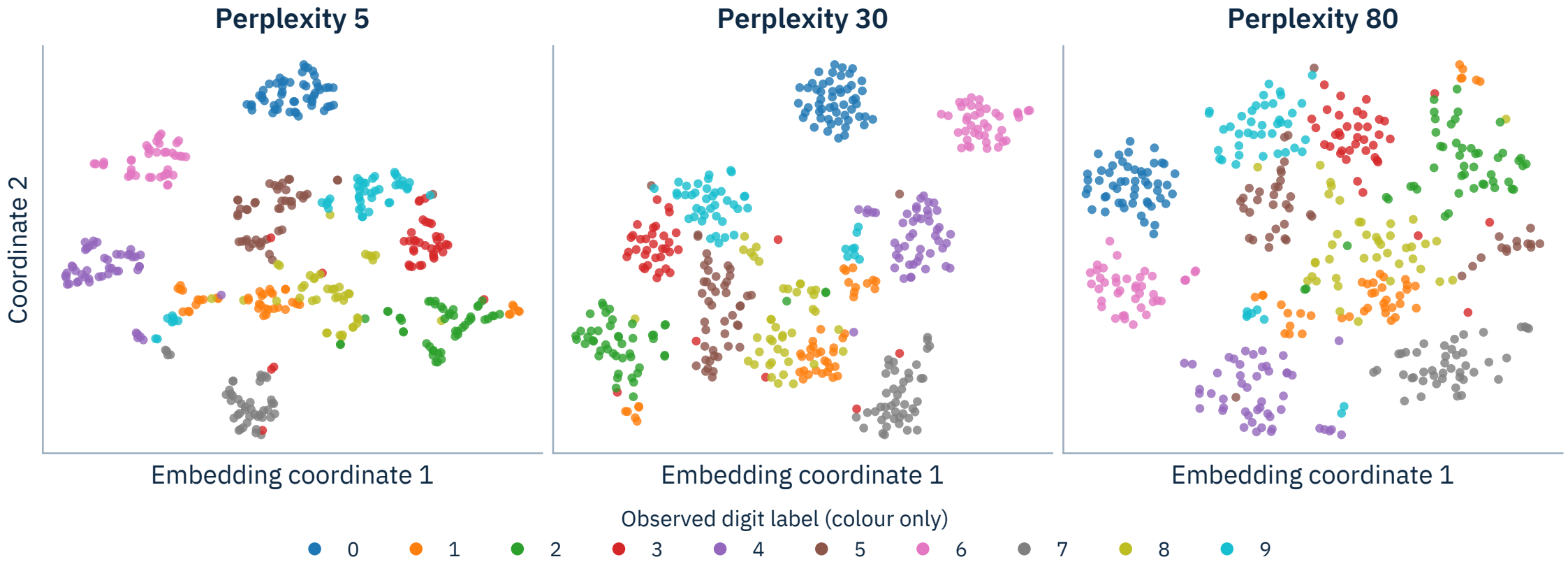
◆ Perplexity is roughly the effective number of neighbours, and must be smaller than the number of observations.

The t-SNE objective is asymmetric

$$\mathrm{KL}(P\|Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

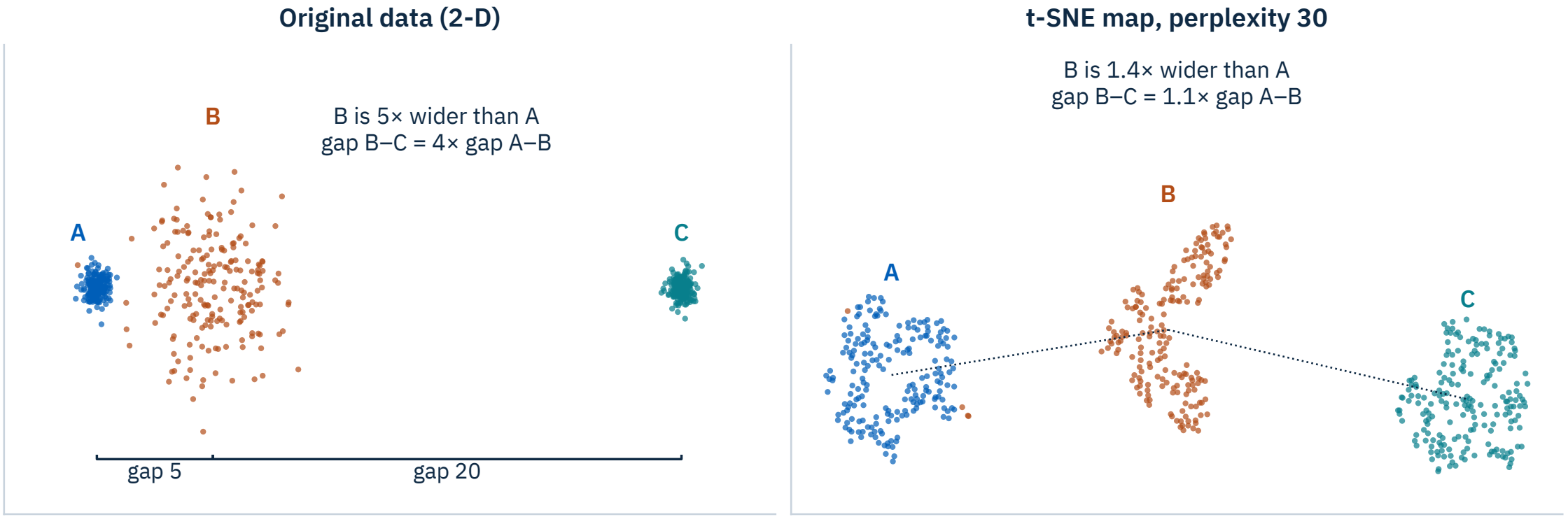
- P describes high-dimensional similarities
- Q describes similarities in the embedding
- Missing an important original neighbour incurs a strong penalty

Perplexity changes the map



t-SNE fits on the same standardized digit subset with a fixed seed. Colours show known digit labels for inspection only.

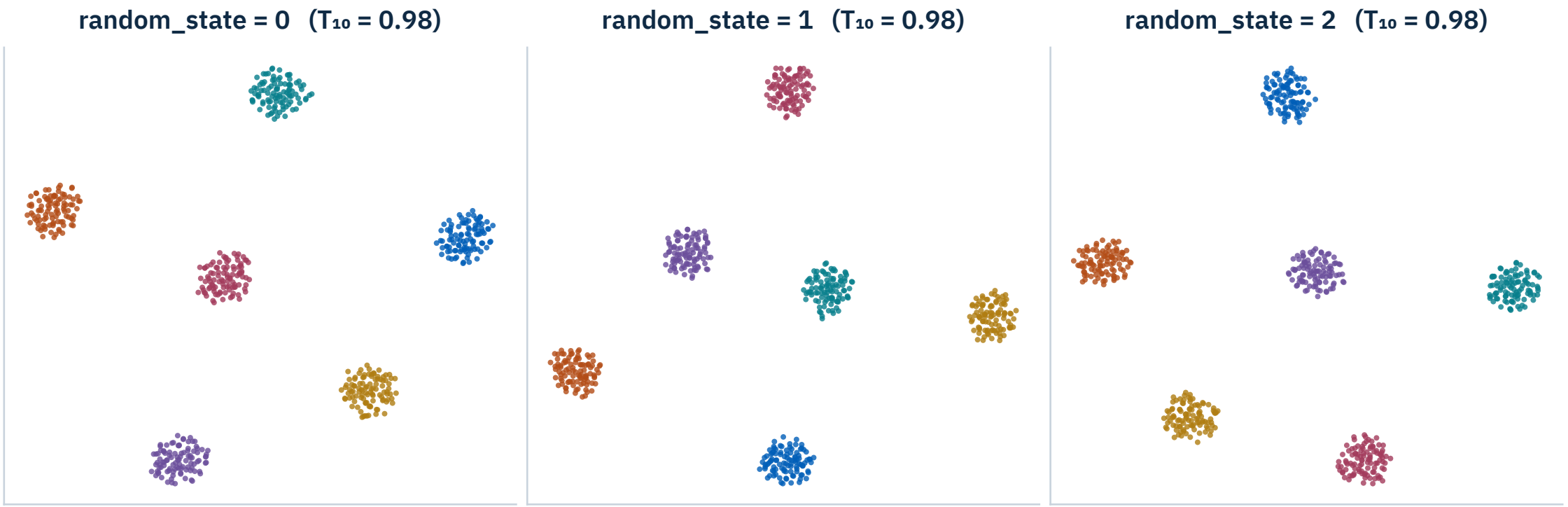
Island sizes and gaps in t-SNE are not measurements



Synthetic 2-D clusters (seed 0): spreads 0.3, 1.5 and 0.3 per axis, centres at 0, 5 and 25; t-SNE seed 0. Spread = RMS distance to the cluster centre.

Random initialization changes the layout, not the neighbours

Same data and perplexity; only random_state differs. Trustworthiness T_{10} is 0.98 in all three.



Six synthetic 10-D Gaussian clusters (make_blobs seed 11); t-SNE perplexity 30, init="random". Colours show generating clusters, not used in fitting.

One t-SNE map supports some claims, not others

✓ Supported by one map

- These points are neighbours in this fit
- Labels tend to occupy nearby areas
- A candidate group is worth checking

vs

✗ Not supported by one map

- Axis values or directions carry units
- Island area or density matches the data
- Gap length measures dissimilarity

◆ Test suspected groups back in the original feature space.

Current t-SNE code uses max_iter, not n_iter

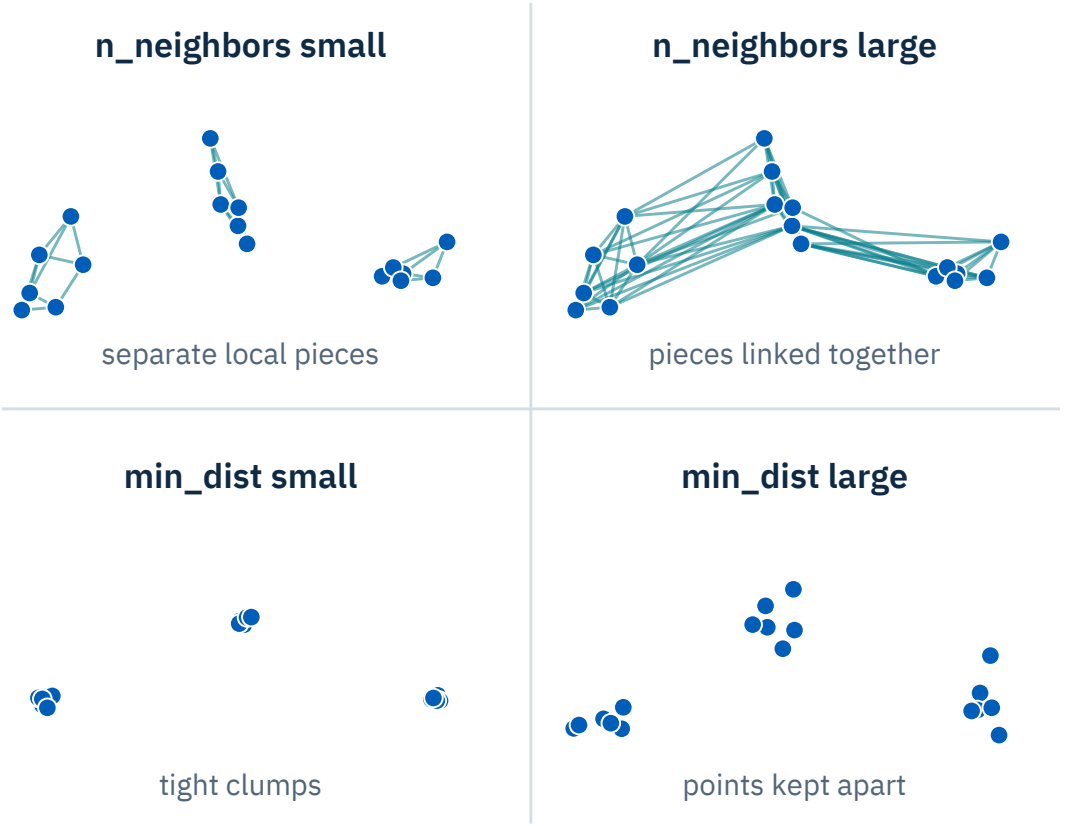
```
from sklearn.manifold import TSNE

embedding = TSNE(
    n_components=2, perplexity=30,
    init="pca", learning_rate="auto",
    max_iter=1000, random_state=42
).fit_transform(Z)
```

Z is an explicitly prepared feature matrix with more than 30 rows.

UMAP starts from a neighbourhood graph

- Build weighted local relationships in the input space
- Optimize a layout that approximates those relationships
- n_neighbors sets the neighbourhood scale of the graph
- min_dist sets how tightly points may pack in the map



Schematic, not UMAP output; umap-learn is an optional package (requirements-labs-optional.txt).

Graph cross-entropy balances attraction and separation

$$-\sum_{i<j}[v_{ij}\log w_{ij}+(1-v_{ij})\log(1-w_{ij})]$$

- v represents input-graph membership strength
- w represents membership implied by embedded distance
- Practical optimization uses sampling rather than all pairs on every step

UMAP supports an explicit future-data transform

```python
import umap

reducer = umap.UMAP(
    n_neighbors=15, min_dist=0.1,
    metric="euclidean", random_state=42
)
Z_train = reducer.fit_transform(X_train_scaled)
Z_new = reducer.transform(X_new_scaled)
```

Evaluate whether the fitted representation transfers to new wells.

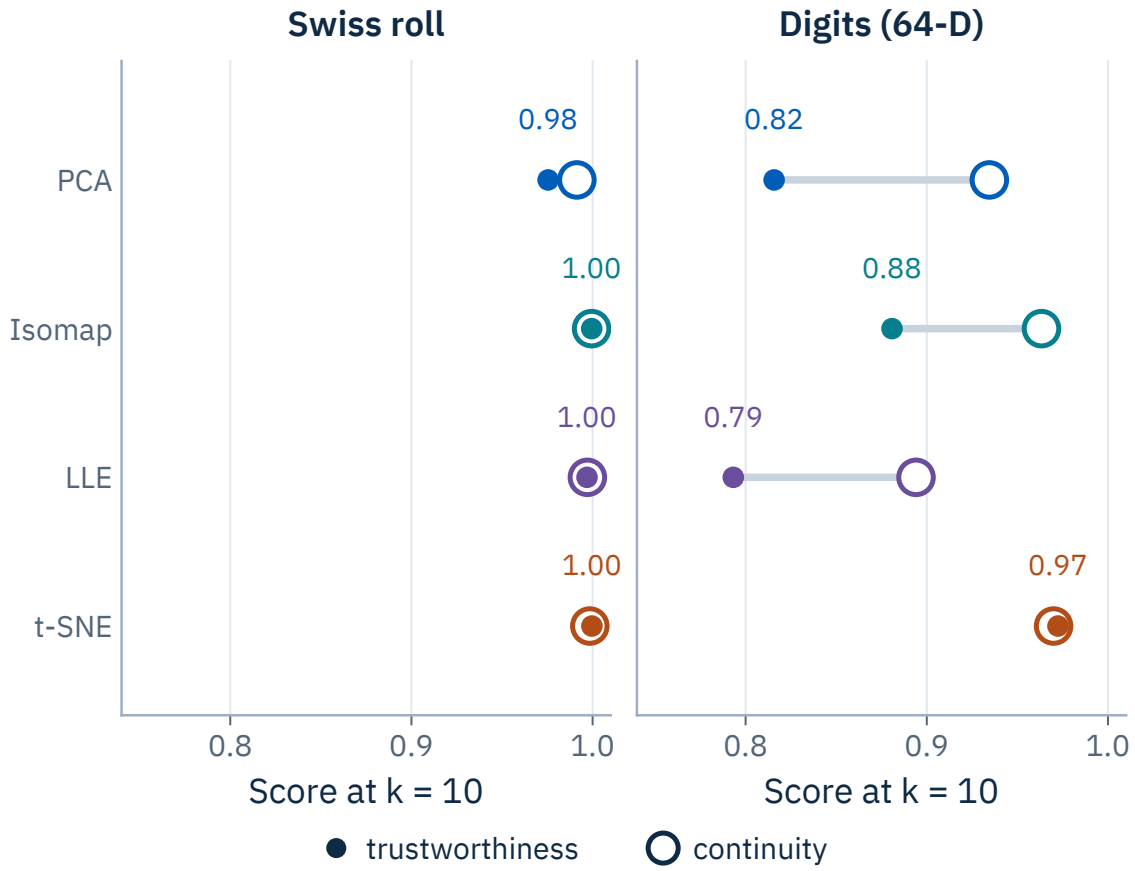
Supervised UMAP changes the evidence

- Supplying labels influences the fitted representation
- Fit it inside training folds when predicting those labels
- Evaluate on separate observations whose labels did not shape the map
- Label a supervised visualization clearly in presentations

⚠ A map shaped by the labels cannot also be the evidence that those labels separate.

Trustworthiness scores how faithful neighbourhoods are

- Trustworthiness: are map neighbours true neighbours?
- Continuity: are true neighbours kept in the map?
- Swiss roll: PCA 0.98, the other methods 0.997 or more
- Digits: t-SNE 0.97, Isomap 0.88, PCA 0.82, LLE 0.79



sklearn.manifold.trustworthiness at k = 10; continuity is the same function with the arguments swapped. Swiss roll seed 0; 600 standardized digit images (seed 42).

A comparison needs a fixed protocol

Dimension	PCA	t-SNE	UMAP
Geometry	Linear variance	Local similarities	Neighbourhood graph
Interpretation	Directions and loadings	Local visual neighbourhoods	Local visual neighbourhoods
Reproducibility	Usually stable up to sign	Check seeds and settings	Check seeds and settings
Validation	Reconstruction or task score	Neighbourhoods and stability	Neighbourhoods, stability, task score

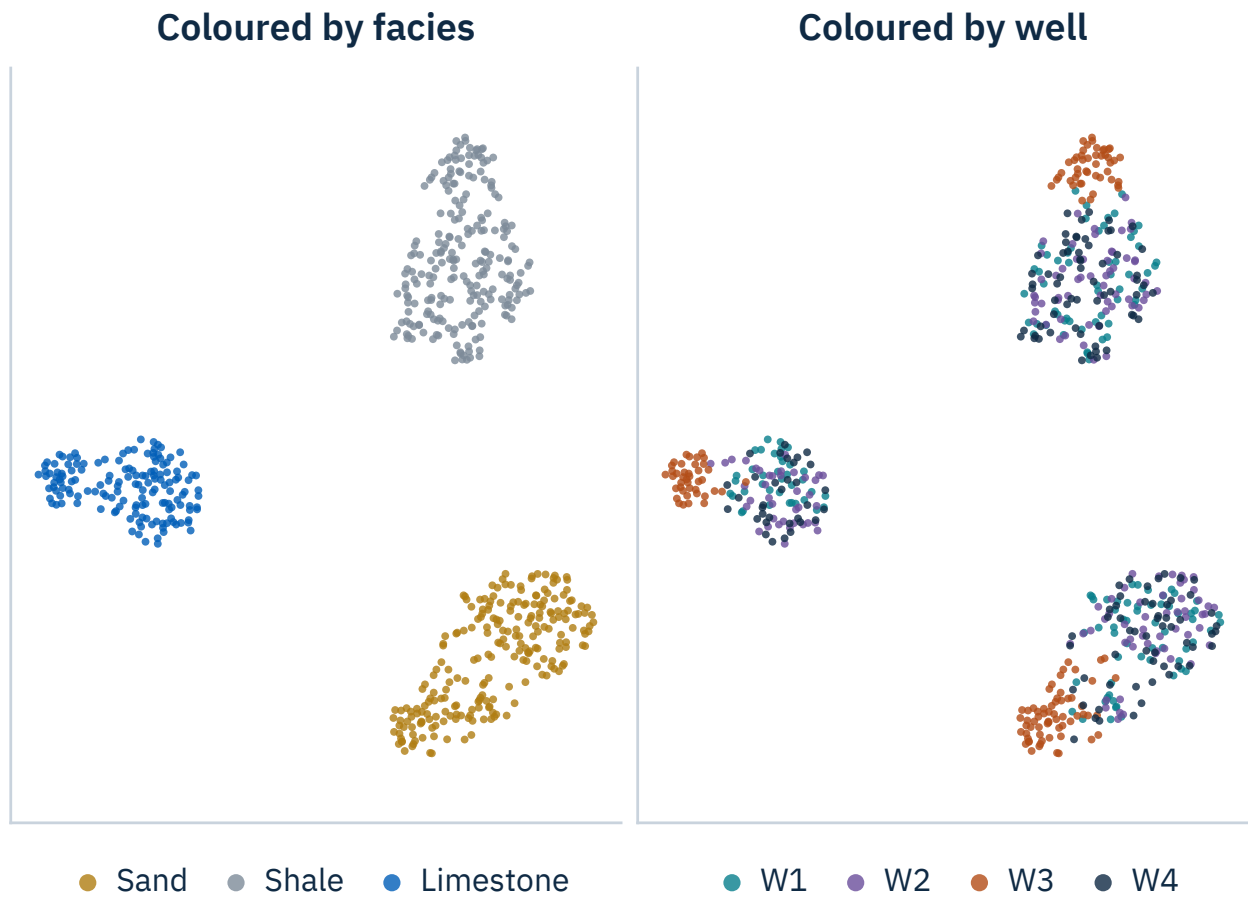
◆ Same rows, same scaling, recorded settings and at least two seeds for every method.

PRACTICE

5 min · Pairs

Colour the map by metadata before trusting it

1. What does the left panel suggest?
2. What does the right panel add?
3. Which check in the original logs comes next?
4. What would you change before clustering?



Synthetic logs from four wells (seed 21); t-SNE on standardized GR, RHOB, NPHI and PEF.

LABORATORY

60 min

Lab 2: nonlinear maps and a fair comparison

- Route A (02_tsne): production data; perplexity 5 to 50; two seeds with init="random"
- Route B (04_comparison): PCA and t-SNE on petro.csv under one protocol
- Add `trustworthiness(X, emb, n_neighbors=10)` to every map
- Colour by metadata after fitting; write one supported and one unsupported claim
- UMAP (03_umap) is optional: umap-learn is in requirements-labs-optional.txt

NOTEBOOK

`exercises/02_tsne/tSNE_Production_Student.ipynb`

EVIDENCE TO SUBMIT

- Maps at two perplexities and two seeds, each with its trustworthiness
- One supported conclusion, one unsupported interpretation, one next check

DISCUSSION

Lab 2 debrief: maps differ more than their scores do

Values computed from the course files with the notebooks' settings:

0.91 vs 0.70

petro.csv trustworthiness (k = 10): t-SNE vs PCA

0.97–0.98

production t-SNE trustworthiness at perplexity 5 to 50

0.31 vs 0.06

facies silhouette: log_0 and log_1 only vs all ten logs

- Pictures changed a lot with settings; neighbourhood scores barely moved. The facies signal lives in two logs.

PART 6 OF 6

06

Reduction in practice

Choose by goal; check results in original units.

- A goal-to-method guide
- Drilling: compression error per channel
- Logs: reconstruction error for quality control
- Production: shape versus amplitude

Choose the method from the goal

Goal	First choice	Evidence to report
Keep original sensors	Filter, wrapper or embedded selection	Selection frequency across folds
Predict a target	Selection or PCA inside the pipeline	Held-out score vs. no reduction
Compress or QC	PCA, k by held-out error	Error per channel in original units
Visualize neighbourhoods	t-SNE or UMAP, with PCA alongside	Trustworthiness, seeds, metadata colours
Unmix or find parts	ICA or NMF	Physical plausibility of components

Compression error must be read channel by channel

- With k = 3, most channels keep a quarter to a third of their SD as error
- Gas is lost completely at k = 3 (8.5 units, its full SD)
- k = 4 recovers gas: the channel that matters may carry little variance

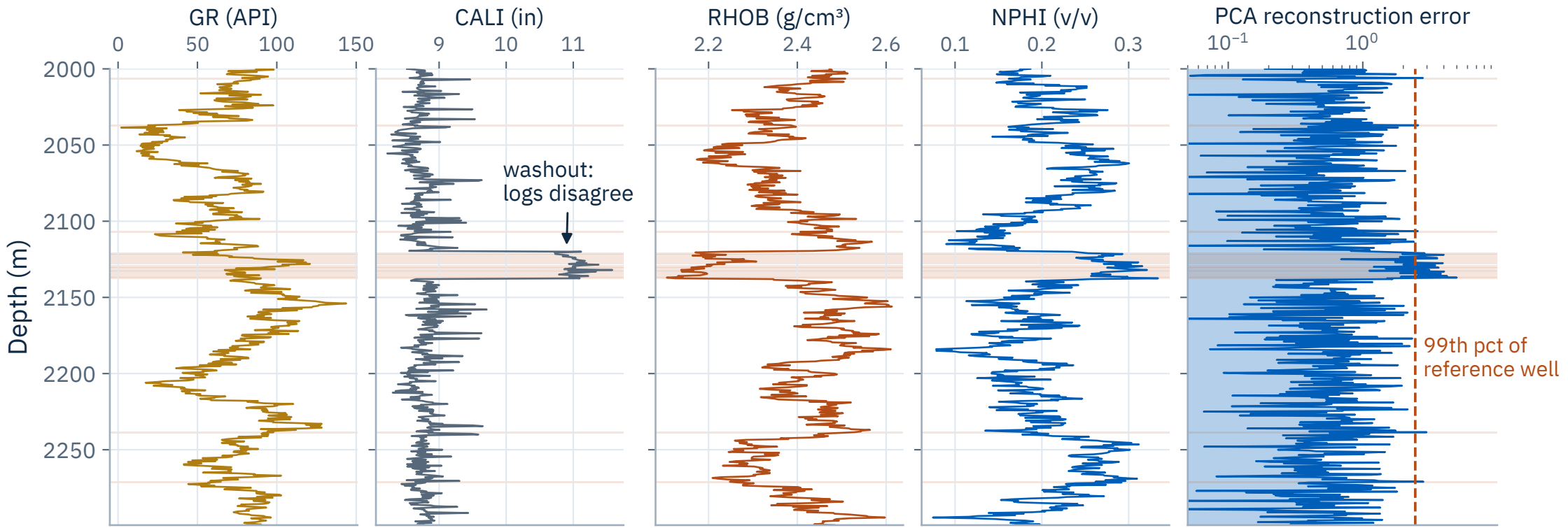
	k = 1	k = 2	k = 3	k = 4	k = 5
WOB (klbf)	1.8	1.6	1.3	1.3	1.3
Torque (kft·lbf)	1.9	1.9	0.8	0.8	0.8
RPM (rpm)	24	25	4.8	3.1	3.1
ROP (ft/h)	11	11	3.9	3.7	3.4
SPP (psi)	242	95	62	56	51
Flow (gpm)	55	20	16	14	14
ECD (ppg)	0.10	0.04	0.03	0.03	0.02
DiffP (psi)	50	33	25	25	16
Hookload (klbf)	2.4	2.2	2.0	2.0	1.9
Gas (units)	8.2	8.3	8.5	0.08	0.07

Validation RMSE in engineering units (shade = RMSE / channel SD)

Synthetic drilling channels (seed 4); PCA fitted on standardized minutes 0–429; validation RMSE on minutes 430–719 in original units.

Reconstruction error screens logs for bad-hole intervals

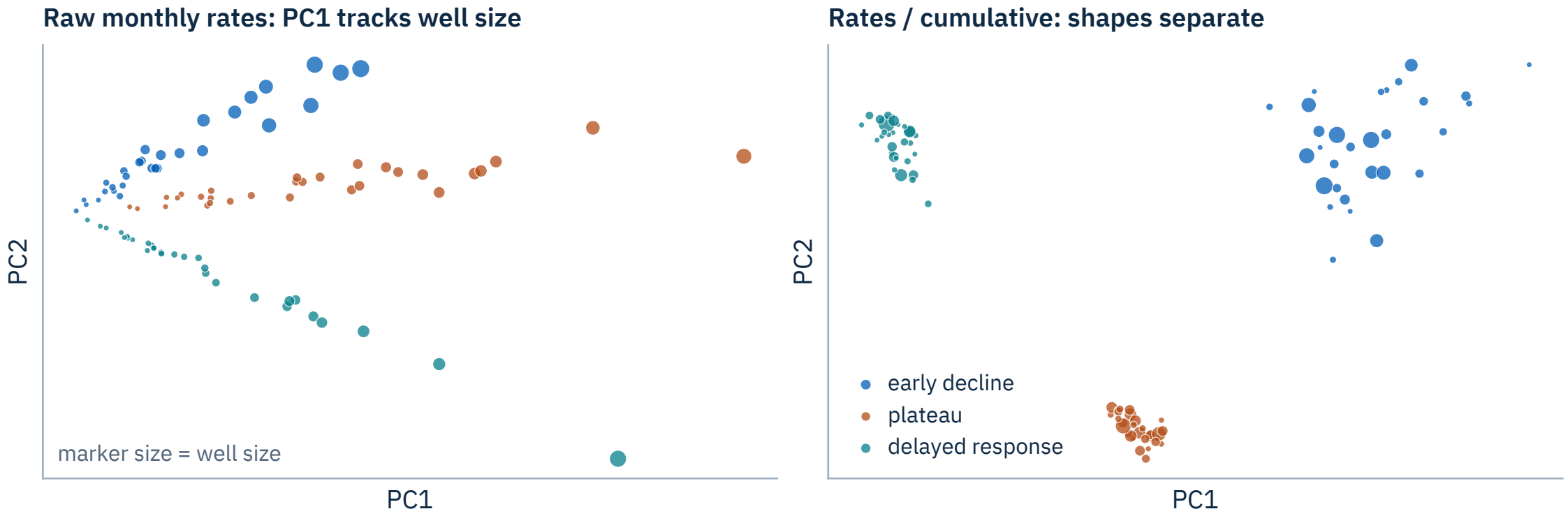
PCA fitted on a reference well; every depth of a new well is scored by its reconstruction error.



Synthetic log suites (reference seed 11, new well seed 23) with an injected washout at 2,120–2,138 m; PCA with k = 4 on the standardized reference well; shading marks depths above the threshold.

Production maps depend on shape versus amplitude

Divide each well's rates by its cumulative before PCA when shape, not size, defines similarity.



Synthetic production profiles for 90 wells (seed 13): three shape families, well sizes varying more than tenfold; colours added after fitting.

REDUCTION IN PRACTICE

Energy applications need original-unit checks

Well-log suites

- Does component structure follow lithology or tool effects?

Seismic attributes

- Do neighbourhoods survive changes in scaling and sampling?

Drilling sensors

- Operating regimes, or sensor outages and calibration drift?

Production profiles

- Does amplitude or shape define similarity?

◆ Colour maps by well, campaign or tool after fitting; never call an island a compartment without evidence.

KEY TAKEAWAYS

Reduction must earn its place in the workflow

- ✓ High dimensions dilute distances; reduce for a stated goal, not by habit.
- ✓ Select features inside cross-validation, and report how stable the choice is.
- ✓ Standardize mixed units; choose k by task score or held-out error, not a fixed 95%.
- ✓ Kernel PCA, LDA, ICA and NMF change the objective, not just the geometry.
- ✓ Read t-SNE and UMAP maps for neighbours only; check trustworthiness, seeds and metadata.
- ✓ Check reconstructions and maps in original units, per channel and per well.

PRACTICE

5 min · Individual

Five questions check today's core ideas

1. Does retaining 95% of variance guarantee retaining target information?
2. Why must SelectKBest sit inside the cross-validation pipeline?
3. Can distances between t-SNE islands be read as physical distances?
4. What must stay fixed when transforming a new well with UMAP?
5. Name one check that a 2-D map's neighbourhoods are trustworthy.

References and further study

- scikit-learn User Guide: Feature selection; Decomposing signals in components (PCA, kernel PCA, ICA, NMF); Manifold learning; Linear and quadratic discriminant analysis.
- Guyon and Elisseeff (2003), An introduction to variable and feature selection, JMLR 3. Hastie, Tibshirani and Friedman, The Elements of Statistical Learning, 2nd ed., sections 2.5 and 7.10.2, chapter 14.
- Jolliffe and Cadima (2016), Principal component analysis: a review and recent developments, Phil. Trans. R. Soc. A 374.
- Tenenbaum, de Silva and Langford (2000), Science 290; Roweis and Saul (2000), Science 290; van der Maaten and Hinton (2008), Visualizing data using t-SNE, JMLR 9; McInnes, Healy and Melville (2018), UMAP, arXiv:1802.03426.
- Hyvärinen and Oja (2000), Independent component analysis, Neural Networks 13; Lee and Seung (1999), Nature 401; Wattenberg, Viégas and Johnson (2016), How to use t-SNE effectively, Distill.